```
@RunWith(AndroidJUnit4::class)
class ExampleInstrumentedTest {
    @Test
    fun useAppContext() {
        val appContext = InstrumentationRegistry.getInstrumentation().targetContext
        assertEquals("com.im.qx", appContext.packageName)
class ExampleUnitTest {
    @Test
    fun addition_isCorrect() {
        assertEquals(4, 2 + 2)
class MainActivity : WKBaseActivity<ActivityMainBinding>() {
    override fun getViewBinding(): ActivityMainBinding {
        return ActivityMainBinding.inflate(layoutInflater)
    override fun initView() {
        super.initView()
        val isShowDialog: Boolean =
            WKSharedPreferencesUtil.getInstance().getBoolean("show_agreement_dialog")
        if (isShowDialog) {
            showDialog()
        } else gotoApp()
    private fun gotoApp() {
        if (!TextUtils.isEmpty(WKConfig.getInstance().token)) {
            if (TextUtils.isEmpty(WKConfig.getInstance().userInfo.name)) {
                startActivity(Intent(this@MainActivity, PerfectUserInfoActivity::c...
            } else {
                val publicRSAKey: String =
                    WKIM.getInstance().cmdManager.rsaPublicKey
                if (TextUtils.isEmpty(publicRSAKey)) {
                    val intent = Intent(this@MainActivity, WKLoginActivity::class....
                    intent.putExtra("from", getIntent().getIntExtra("from", 0))
                    startActivity(intent)
                } else {
                    startActivity(Intent(this@MainActivity, TabActivity::class.java))
        } else {
            val intent = Intent(this@MainActivity, WKLoginActivity::class.java)
            intent.putExtra("from", getIntent().getIntExtra("from", 0))
            startActivity(intent)
        finish()
    private fun showDialog() {
        val content = getString(R.string.dialog_content)
        val linkSpan = SpannableStringBuilder()
        linkSpan.append(content)
        val userAgreementIndex = content.indexOf(getString(R.string.main_user_agre...
        linkSpan.setSpan(
            NormalClickableSpan(
                true,
                ContextCompat.getColor(this, R.color.blue),
                NormalClickableContent(NormalClickableContent.NormalClickableTypes...
                object : NormalClickableSpan.IClick {
                    override fun onClick(view: View) {
```

```
                showWebView(
                    WKApiConfig.baseWebUrl + "user_agreement.html"
                )
            }), userAgreementIndex, userAgreementIndex + 6, Spannable.SPAN_EXC...
        )
        val privacyPolicyIndex = content.indexOf(getString(R.string.main_privacy_p...
        linkSpan.setSpan(
            NormalClickableSpan(true,
                ContextCompat.getColor(this, R.color.blue),
                NormalClickableContent(NormalClickableContent.NormalClickableTypes...
                object : NormalClickableSpan.IClick {
                    override fun onClick(view: View) {
                        WKApiConfig.baseWebUrl + "privacy_policy.html"
                }), privacyPolicyIndex, privacyPolicyIndex + 6, Spannable.SPAN_EXC...
        )
        WKDialogUtils.getInstance().showDialog(
            this,
            getString(R.string.dialog_title),
            linkSpan,
            false,
            getString(R.string.disagree),
            getString(R.string.agree),
            0,
            0
        ) { index ->
            if (index == 1) {
                WKSharedPreferencesUtil.getInstance()
                    .putBoolean("show_agreement_dialog", false)
                WKBaseApplication.getInstance().init(
                    WKBaseApplication.getInstance().packageName,
                    WKBaseApplication.getInstance().application
                )
                gotoApp()
            } else {
                finish()
class KeepAliveService : Service() {
    companion object {
        private const val CHANNEL_ID = "ts_keep_alive"
        private const val CHANNEL_NAME = "后台运行"
        private const val NOTIFICATION_ID = 10086
        fun start(context: Context) {
            val intent = Intent(context, KeepAliveService::class.java)
            try {
                if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                    context.startForegroundService(intent)
                } else {
                    context.startService(intent)
            } catch (_: Throwable) {
        fun stop(context: Context) {
            try {
```

```
            context.stopService(Intent(context, KeepAliveService::class.java))
        } catch (_: Throwable) {
    override fun onBind(intent: Intent?): IBinder? = null
    override fun onCreate() {
        super.onCreate()
        startForeground(NOTIFICATION_ID, buildNotification())
    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        startForeground(NOTIFICATION_ID, buildNotification())
        return START_STICKY
    override fun onTaskRemoved(rootIntent: Intent?) {
        val restartIntent = Intent(applicationContext, KeepAliveService::class.java)
        try {
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                applicationContext.startForegroundService(restartIntent)
            } else {
                applicationContext.startService(restartIntent)
        } catch (_: Throwable) {
        super.onTaskRemoved(rootIntent)
    private fun buildNotification(): Notification {
        ensureChannel()
        val appName = try {
            applicationContext.getString(applicationContext.applicationInfo.labelRes)
        } catch (_: Throwable) {
            "IM"
        val clickIntent = Intent(this, TabActivity::class.java).apply {
            flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_SINGLE_TOP
        val pendingIntent = PendingIntent.getActivity(
            this,
            0,
            clickIntent,
            PendingIntent.FLAG_IMMUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
        )
        return NotificationCompat.Builder(this, CHANNEL_ID)
            .setSmallIcon(R.mipmap.ic_logo)
            .setColor(0xFF007BF9.toInt())
            .setContentTitle("$appName 正在后台运行")
            .setContentText("用于持续接收新消息通知")
            .setPriority(NotificationCompat.PRIORITY_MIN)
            .setCategory(NotificationCompat.CATEGORY_SERVICE)
            .setOngoing(true)
            .setShowWhen(false)
            .setContentIntent(pendingIntent)
            .build()
    private fun ensureChannel() {
        if (Build.VERSION.SDK_INT < Build.VERSION_CODES.O) return
        val manager = getSystemService(Context.NOTIFICATION_SERVICE) as Notificati...
        if (manager.getNotificationChannel(CHANNEL_ID) != null) return
        val channel = NotificationChannel(
            CHANNEL_ID,
            CHANNEL_NAME,
```

```
        NotificationManager.IMPORTANCE_MIN
    ).apply {
        description = "用于持续接收新消息通知，关闭后将无法在后台及时收到消息"
        setShowBadge(false)
        enableLights(false)
        enableVibration(false)
        setSound(null, null)
        lockscreenVisibility = Notification.VISIBILITY_SECRET
    manager.createNotificationChannel(channel)
public class AppFrontBackHelper {
  private OnAppStatusListener mOnAppStatusListener;
  public AppFrontBackHelper() {
  public void register(Application application, OnAppStatusListener listener) {
    mOnAppStatusListener = listener;
    application.registerActivityLifecycleCallbacks(activityLifecycleCallbacks);
  public void unRegister(Application application) {
    application.unregisterActivityLifecycleCallbacks(activityLifecycleCallbacks);
  private final Application.ActivityLifecycleCallbacks activityLifecycleCallback...
    private int activityStartCount = 0;
    @Override
    public void onActivityCreated(Activity activity, Bundle savedInstanceState) {
    @Override
    public void onActivityStarted(Activity activity) {
      activityStartCount++;
      if (activityStartCount == 1) {
        if (mOnAppStatusListener != null) {
          mOnAppStatusListener.onFront();
    @Override
    public void onActivityResumed(Activity activity) {
    @Override
    public void onActivityPaused(Activity activity) {
    @Override
    public void onActivityStopped(Activity activity) {
      activityStartCount--;
      if (activityStartCount == 0) {
        if (mOnAppStatusListener != null) {
          mOnAppStatusListener.onBack();
    @Override
    public void onActivitySaveInstanceState(Activity activity, Bundle outState) {
    @Override
    public void onActivityDestroyed(Activity activity) {
  public interface OnAppStatusListener {
    void onFront();
    void onBack();
class TSApplication : MultiDexApplication() {
  override fun onCreate() {
    super.onCreate()
    val processName = getProcessName(this, Process.myPid())
    if (processName != null) {
      val defaultProcess = processName == getAppPackageName()
```

```
        if (defaultProcess) {
            initAll()
        registerActivityLifecycleCallbacks(object : ActivityLifecycleCallbacks {
            override fun onActivityCreated(p0: Activity, p1: Bundle?) {
            override fun onActivityStarted(p0: Activity) {
            override fun onActivityResumed(p0: Activity) {
                ActManagerUtils.getInstance().currentActivity = p0
            override fun onActivityPaused(p0: Activity) {
            override fun onActivityStopped(p0: Activity) {
            override fun onActivitySaveInstanceState(p0: Activity, p1: Bundle) {
            override fun onActivityDestroyed(p0: Activity) {
        })
    override fun onConfigurationChanged(newConfig: Configuration) {
        super.onConfigurationChanged(newConfig)
        if (applicationContext != null && applicationContext.resources != null && ...
            WKMultiLanguageUtil.getInstance().setConfiguration()
            Theme.applyTheme()
            killAppProcess()
    private fun killAppProcess() {
        ActManagerUtils.getInstance().clearAllActivity()
        Process.killProcess(Process.myPid())
        exitProcess(0)
    override fun attachBaseContext(base: Context?) {
        super.attachBaseContext(WKMultiLanguageUtil.getInstance().attachBaseContex...
    private fun initAll() {
        WKMultiLanguageUtil.getInstance().init(this)
        WKBaseApplication.getInstance().init(getAppPackageName(), this)
        Theme.applyTheme()
        initApi()
        WKLoginApplication.getInstance().init(this)
        WKScanApplication.getInstance().init(this)
        WKUIKitApplication.getInstance().init(this)
        WKPushApplication.getInstance().init(getAppPackageName(), this)
        WKGroupManageApplication.getInstance().init();
        WKVideoApplication.getInstance().init(this);
        WKFileApplication.getInstance().init(this);
        WKKeepAliveApplication.instance.init();
        addAppFrontBack()
        addListener()
        if (!TextUtils.isEmpty(WKConfig.getInstance().token)) {
            KeepAliveService.start(this)
            Handler(Looper.getMainLooper()).postDelayed({
                WKUIKitApplication.getInstance().checkBanStatusAndHandle()
            }, 2000)
    private fun initApi() {
        var apiURL = WKSharedPreferencesUtil.getInstance().getSP("api_base_url")
        if (TextUtils.isEmpty(apiURL)) {
            apiURL = "https:
            WKApiConfig.initBaseURL(apiURL)
        } else {
```

```
        WKApiConfig.initBaseURLIncludeIP(apiURL)
    private fun getAppPackageName(): String {
        return packageName
    private fun getProcessName(cxt: Context, pid: Int): String? {
        val am = cxt.getSystemService(ACTIVITY_SERVICE) as ActivityManager
        val runningApps = am.runningAppProcesses ?: return null
        for (app in runningApps) {
            if (app.pid == pid) {
                return app.processName
        return null
    private fun addAppFrontBack() {
        val helper = AppFrontBackHelper()
        helper.register(this, object : AppFrontBackHelper.OnAppStatusListener {
            override fun onFront() {
                if (!TextUtils.isEmpty(WKConfig.getInstance().token)) {
                    Handler(Looper.getMainLooper()).postDelayed({
                        EndpointManager.getInstance()
                            .invoke("chow_check_lock_screen_pwd", null)
                    }, 1000)
                    WKIMUtils.getInstance().initIMListener()
                    WKUIKitApplication.getInstance().startChat()
                    UserModel.getInstance().getOnlineUsers()
                    KeepAliveService.start(this@TSApplication)
                    WKUIKitApplication.getInstance().checkBanStatusAndHandle()
                    checkAppNewVersion()
            override fun onBack() {
                WKSharedPreferencesUtil.getInstance()
                    .putLong("lock_start_time", WKTimeUtils.getInstance().currentS...
        })
    private fun checkAppNewVersion() {
        Handler(Looper.getMainLooper()).postDelayed({
            WKCommonModel.getInstance()
                .getAppNewVersion(false) { version: AppVersion? ->
                    val activity = ActManagerUtils.getInstance().currentActivity
                    if (activity == null || activity.isFinishing) return@getAppNew...
                    if (version == null || TextUtils.isEmpty(version.download_url)...
                    val localV = WKDeviceUtils.getInstance().getVersionName(activity)
                    if (WKDeviceUtils.isRemoteVersionNewer(version.app_version, lo...
                        WKDialogUtils.getInstance().showNewVersionDialog(activity,...
        }, 1500)
    private fun addListener() {
        createNotificationChannel()
        EndpointManager.getInstance().setMethod(
            "ts_keep_alive_on_login",
            EndpointCategory.loginMenus
        ) { LoginMenu { KeepAliveService.start(this@TSApplication) } }
        EndpointManager.getInstance().setMethod(
            "ts_send_message_receipt", EndpointSID.sendMessage
        ) { obj ->
            if (obj is WKSendMsgMenu) {
```

```
            obj.option.setting.receipt =
                if (obj.channel.channelType == WKChannelType.PERSONAL) 1 else 0
            null
        EndpointManager.getInstance().setMethod("read_msg") { obj ->
            if (obj is ReadMsgMenu && obj.channelType == WKChannelType.PERSONAL) {
                WKCommonModel.getInstance().readMsg(obj.channelID, obj.channelType...
            null
        EndpointManager.getInstance().setMethod("main_show_home_view") { `object` ->
            KeepAliveService.stop(this@TSApplication)
            if (`object` != null) {
                val from = `object` as Int
                val intent = Intent(applicationContext, MainActivity::class.java)
                intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP or Intent.FLAG_ACTI...
                intent.putExtra("from", from)
                startActivity(intent)
            null
        EndpointManager.getInstance().setMethod("show_tab_home") {
            val intent = Intent(applicationContext, TabActivity::class.java)
            intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK
            startActivity(intent)
            null
        EndpointManager.getInstance().setMethod("play_new_msg_Media") {
            WKPlaySound.getInstance().playRecordMsg(R.raw.newmsg)
            null
    private fun createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val notificationManager = applicationContext.getSystemService(
                NotificationManager::class.java
            )
            runCatching {
                notificationManager.deleteNotificationChannel(WKConstants.legacyNe...
                notificationManager.deleteNotificationChannel(WKConstants.legacyNe...
            val name: CharSequence = applicationContext.getString(R.string.new_msg...
            val description = applicationContext.getString(R.string.new_msg_notifi...
            val channel = NotificationChannel(
                WKConstants.newMsgChannelID,
                name,
                NotificationManager.IMPORTANCE_HIGH
            )
            channel.description = description
            channel.enableVibration(true)
            channel.lockscreenVisibility = Notification.VISIBILITY_PUBLIC
            channel.setSound(
                Uri.parse(ContentResolver.SCHEME_ANDROID_RESOURCE + ":
                Notification.AUDIO_ATTRIBUTES_DEFAULT
            )
            notificationManager.createNotificationChannel(channel)
        createNotificationRTCChannel()
    private fun createNotificationRTCChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
```

```
        val name: CharSequence = applicationContext.getString(R.string.new_rtc...
        val description = applicationContext.getString(R.string.new_rtc_notifi...
        val channel = NotificationChannel(
          WKConstants.newRTCChannelID,
          name,
          NotificationManager.IMPORTANCE_HIGH
        )
        channel.description = description
        channel.enableVibration(true)
        channel.vibrationPattern = longArrayOf(0, 100, 100, 100, 100, 100)
        channel.lockscreenVisibility = Notification.VISIBILITY_PUBLIC
        channel.setSound(
          Uri.parse(ContentResolver.SCHEME_ANDROID_RESOURCE + ":
          Notification.AUDIO_ATTRIBUTES_DEFAULT
        )
        val notificationManager = applicationContext.getSystemService(
          NotificationManager::class.java
        )
        notificationManager.createNotificationChannel(channel)
@RunWith(AndroidJUnit4.class)
public class ExampleInstrumentedTest {
  @Test
  public void useAppContext() {
    Context appContext = InstrumentationRegistry.getInstrumentation().getTarge...
    assertEquals("com.chat.base.test", appContext.getPackageName());
public class ExampleUnitTest {
  @Test
  public void addition_isCorrect() {
    assertEquals(4, 2 + 2);
public class WKBaseApplication {
  private WeakReference<Context> context;
  private DBHelper mDbHelper;
  private String fileDir = "wkIM";
  public boolean disconnect = true;
  public String versionName;
  public String appID = "wukongchat";
  public static volatile Handler applicationHandler;
  private WKBaseApplication() {
  private static class WApplicationBinder {
    final static WKBaseApplication wb = new WKBaseApplication();
  public static WKBaseApplication getInstance() {
    return WApplicationBinder.wb;
  public String packageName;
  public Application application;
  private List<AppModule> appModules;
  public void init(@NonNull String packageName, Application context) {
    applicationHandler = new Handler(context.getMainLooper());
    this.packageName = packageName;
    this.application = context;
    this.context = new WeakReference<>(context);
```

```
    float density = context.getResources().getDisplayMetrics().density;
    AndroidUtilities.setDensity(density);
    boolean isShowDialog = WKSharedPreferencesUtil.getInstance().getBoolean("s...
    if (isShowDialog) {
        return;
    String json = WKSharedPreferencesUtil.getInstance().getSPWithUID("app_modu...
    if (!TextUtils.isEmpty(json)) {
        appModules = JSON.parseArray(json, AppModule.class);
    versionName = WKDeviceUtils.getInstance().getVersionName(context);
    Glide.get(context).getRegistry().replace(GlideUrl.class, InputStream.class...
    initCacheDir();
    new Thread(() -> {
        EmojiManager.getInstance().init();
        LottieUtils.init(context);
        CrashHandler.getInstance().init(context);
    }).start();
    EndpointManager.getInstance().setMethod("play_video", object -> {
        if (object instanceof PlayVideoMenu playVideoMenu) {
            @SuppressWarnings("unchecked") ActivityOptionsCompat activityOptio...
            Intent intent = new Intent(playVideoMenu.activity, PlayVideoActivi...
            intent.putExtra("coverImg", playVideoMenu.coverUrl);
            intent.putExtra("url", playVideoMenu.playUrl);
            intent.putExtra("title", playVideoMenu.videoTitle);
            intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
            playVideoMenu.activity.startActivity(intent, activityOptions.toBun...
            playVideoMenu.activity.overridePendingTransition(R.anim.fade_in, R...
        return null;
    });
public Context getContext() {
    return context.get();
public synchronized DBHelper getDbHelper() {
    if (mDbHelper == null) {
        String uid = WKConfig.getInstance().getUid();
        if (!TextUtils.isEmpty(uid) && context != null && context.get() != nul...
            mDbHelper = DBHelper.getInstance(context.get(), uid);
    return mDbHelper;
public void closeDbHelper() {
    if (mDbHelper != null) {
        mDbHelper.close();
        mDbHelper = null;
public String getFileDir() {
    if (TextUtils.isEmpty(fileDir))
        fileDir = "wkIM";
    if (!TextUtils.isEmpty(WKConfig.getInstance().getUid())) {
        fileDir = String.format("%s/%s", fileDir, WKConfig.getInstance().getUi...
    return fileDir;
private void initCacheDir() {
    WKConstants.avatarCacheDir = Objects.requireNonNull(getContext().getExtern...
    WKFileUtils.getInstance().createFileDir(WKConstants.avatarCacheDir);
    WKConstants.imageDir = Objects.requireNonNull(getContext().getExternalFile...
```

```
        WKFileUtils.getInstance().createFileDir(WKConstants.imageDir);
        WKConstants.videoDir = Objects.requireNonNull(getContext().getExternalFile...
        WKFileUtils.getInstance().createFileDir(WKConstants.videoDir);
        WKConstants.voiceDir = Objects.requireNonNull(getContext().getExternalFile...
        WKFileUtils.getInstance().createFileDir(WKConstants.voiceDir);
        WKConstants.chatBgCacheDir = Objects.requireNonNull(getContext().getExtern...
        WKFileUtils.getInstance().createFileDir(WKConstants.chatBgCacheDir);
        WKConstants.messageBackupDir = Objects.requireNonNull(getContext().getExte...
        WKFileUtils.getInstance().createFileDir(WKConstants.messageBackupDir);
        WKConstants.chatDownloadFileDir = Objects.requireNonNull(getContext().getE...
    public AppModule getAppModuleWithSid(String sid) {
        AppModule appModule = null;
        if (WKReader.isNotEmpty(appModules)) {
            for (AppModule appModule1 : appModules) {
                if (appModule1.getSid().equals(sid)) {
                    appModule = appModule1;
                    break;
        return appModule;
    public boolean appModuleIsInjection(AppModule appModule) {
        if (appModule == null) {
            return true;
        return appModule.getStatus() != 0 && appModule.getChecked();
public class Theme {
    public static int colorAccount = 0xFFf65835;
    public static int colorAccountDisable = 0x95F65835;
    public static int color999 = 0xFF999999;
    public static int colorCCC = 0xFFCCCCCC;
    public static int pressedColor = 0xff8c9197;
    private static final Paint maskPaint = new Paint(Paint.ANTI_ALIAS_FLAG);
    public static final String LIGHT_MODE = "light";
    public static final String DARK_MODE = "dark";
    public static final String DEFAULT_MODE = "default";
    public static final String wk_theme_pref = "wk_theme_pref";
    public static final int[][] defaultColorsDark = new int[][]{
            new int[]{0xa65f4167, 0xa6171c2f, 0xa6363d4d, 0xa628293b},
            new int[]{0xa66c3407, 0xa6411f05, 0xa61b0d02, 0xa6080401},
            new int[]{0xa63c0b42, 0xa6210324, 0xa6120314, 0xa62f243f},
            new int[]{0xa6645b12, 0xa6463f09, 0xa6353003, 0xa6E8C06E},
            new int[]{0xa6135360, 0xa607333d, 0xa6021a1f, 0xa6000000},
            new int[]{0xa628072c, 0xa61b1346, 0xa64e0b36, 0xa66e0a6e},
            new int[]{0xa60e4805, 0xa64b044c, 0xa6094c4c, 0xa6555404},
            new int[]{0xa6512908, 0xa6590d41, 0xa66e1606, 0xa6060f6e},
            new int[]{0xa6644141, 0xa6595326, 0xa6590526, 0xa61b1d2c},
            new int[]{0xa67d2f58, 0xa6503f27, 0xa6052037, 0xa6201144},
            new int[]{0xa6144260, 0xa6421b13, 0xa611234e, 0xa6213b4a},
            new int[]{0xa64e1455, 0xa61a1242, 0xa63d0a2b, 0xa628242e},
            new int[]{0xa6482002, 0xa6393303, 0xa63d031a, 0xa62e2002},
    public static GradientDrawable getBackground(int color, float radius, int widt...
        GradientDrawable d = new GradientDrawable();
        d.setColor(color);
```

```
    d.setSize(AndroidUtilities.dp(width), AndroidUtilities.dp(height));
    d.setCornerRadius(AndroidUtilities.dp(radius));
    return d;
  public static GradientDrawable getBackground(int color, float radius) {
    GradientDrawable d = new GradientDrawable();
    d.setColor(color);
    d.setCornerRadius(AndroidUtilities.dp(radius));
    return d;
  public static GradientDrawable.Orientation getGradientOrientation(int gradient...
    switch (gradientAngle) {
      case 0:
        return GradientDrawable.Orientation.BOTTOM_TOP;
      case 90:
        return GradientDrawable.Orientation.LEFT_RIGHT;
      case 135:
        return GradientDrawable.Orientation.TL_BR;
      case 180:
        return GradientDrawable.Orientation.TOP_BOTTOM;
      case 225:
        return GradientDrawable.Orientation.TR_BL;
      case 270:
        return GradientDrawable.Orientation.RIGHT_LEFT;
      case 315:
        return GradientDrawable.Orientation.BR_TL;
      default:
        return GradientDrawable.Orientation.BL_TR;
  public static int getPressedColor() {
    int color = pressedColor;
    color = Color.argb(30, Color.red(color), Color.green(color), Color.blue(co...
    return color;
  private static void applyTheme(@NonNull String themePref) {
    switch (themePref) {
      case LIGHT_MODE -> {
        AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_NIGHT...
      case DARK_MODE -> {
        AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_NIGHT...
      default -> {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
          AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_N...
        } else {
          AppCompatDelegate.setDefaultNightMode(AppCompatDelegate.MODE_N...
  public static void applyTheme() {
    String themePref =
        WKSharedPreferencesUtil.getInstance().getSP(Theme.wk_theme_pref, T...
    Theme.applyTheme(themePref);
  public static String getTheme() {
    return WKSharedPreferencesUtil.getInstance().getSP(Theme.wk_theme_pref, Th...
  public static void setTheme(String s) {
    WKSharedPreferencesUtil.getInstance().putSP(Theme.wk_theme_pref, s);
    Theme.applyTheme(s);
```

```
public static boolean isSystemDarkMode(Context context) {
    UiModeManager uiModeManager = (UiModeManager) context.getSystemService(Con...
    return uiModeManager.getNightMode() == UiModeManager.MODE_NIGHT_YES;
public static boolean getDarkModeStatus(Context context) {
    int mode = context.getResources().getConfiguration().uiMode & Configuratio...
    return mode == Configuration.UI_MODE_NIGHT_YES;
public static int getSwitchViewTrackColor() {
    String wk_theme_pref = WKSharedPreferencesUtil.getInstance().getSP(Theme.w...
    if (wk_theme_pref.equals(DARK_MODE)) {
        return 0xFF585858;
    } else
        return Theme.colorCCC;
public static int getSwitchViewThumbColor() {
    String wk_theme_pref = WKSharedPreferencesUtil.getInstance().getSP(Theme.w...
    if (wk_theme_pref.equals(DARK_MODE)) {
        return 0xFF212223;
    } else
        return 0xFFFFFFFF;
public static boolean isDark() {
    String wk_theme_pref = WKSharedPreferencesUtil.getInstance().getSP(Theme.w...
    return wk_theme_pref.equals(DARK_MODE);
private static Drawable ticksSingleDrawable;
private static Drawable ticksDoubleDrawable;
public static Drawable getTicksSingleDrawable() {
    if (ticksSingleDrawable == null) {
        RLottieDrawable drawable = new RLottieDrawable(WKBaseApplication.getIn...
        drawable.setCurrentFrame(drawable.getFramesCount() - 1);
        ticksSingleDrawable = drawable.getCurrent();
    return ticksSingleDrawable;
public static Drawable getTicksDoubleDrawable() {
    if (ticksDoubleDrawable == null) {
        RLottieDrawable drawable = new RLottieDrawable(WKBaseApplication.getIn...
        drawable.setCurrentFrame(drawable.getFramesCount() - 1);
        ticksDoubleDrawable = drawable.getCurrent();
    return ticksDoubleDrawable;
public static RoundTextView getChannelCategoryTV(Context context, String text,...
    RoundTextView roundTextView = new RoundTextView(context);
    roundTextView.setTextSize(12);
    roundTextView.setText(text);
    roundTextView.setTypeface(Typeface.DEFAULT_BOLD);
    roundTextView.setLines(1);
    roundTextView.setPadding(AndroidUtilities.dp(2), 0, AndroidUtilities.dp(2)...
    roundTextView.setTextColor(textColor);
    roundTextView.setBorderColor(borderColor);
    roundTextView.setBackGroundColor(bgColor);
    roundTextView.setAllRadius(3);
    return roundTextView;
public static Drawable createRadSelectorDrawable(int color, int topRad, int bo...
    if (Build.VERSION.SDK_INT >= 21) {
        maskPaint.setColor(0xffffffff);
```

```
        Drawable maskDrawable = new RippleRadMaskDrawable(topRad, bottomRad);
        ColorStateList colorStateList = new ColorStateList(
                new int[][]{StateSet.WILD_CARD},
                new int[]{color}
        );
        return new RippleDrawable(colorStateList, null, maskDrawable);
    } else {
        StateListDrawable stateListDrawable = new StateListDrawable();
        stateListDrawable.addState(new int[]{android.R.attr.state_pressed}, ne...
        stateListDrawable.addState(new int[]{android.R.attr.state_selected}, n...
        stateListDrawable.addState(StateSet.WILD_CARD, new ColorDrawable(0x000...
        return stateListDrawable;
public static class RippleRadMaskDrawable extends Drawable {
    private Path path = new Path();
    private float[] radii = new float[8];
    boolean invalidatePath = true;
    public RippleRadMaskDrawable(float top, float bottom) {
        radii[0] = radii[1] = radii[2] = radii[3] = AndroidUtilities.dp(top);
        radii[4] = radii[5] = radii[6] = radii[7] = AndroidUtilities.dp(bottom);
    public RippleRadMaskDrawable(float topLeft, float topRight, float bottomRi...
        radii[0] = radii[1] = AndroidUtilities.dp(topLeft);
        radii[2] = radii[3] = AndroidUtilities.dp(topRight);
        radii[4] = radii[5] = AndroidUtilities.dp(bottomRight);
        radii[6] = radii[7] = AndroidUtilities.dp(bottomLeft);
    public void setRadius(float top, float bottom) {
        radii[0] = radii[1] = radii[2] = radii[3] = AndroidUtilities.dp(top);
        radii[4] = radii[5] = radii[6] = radii[7] = AndroidUtilities.dp(bottom);
        invalidatePath = true;
        invalidateSelf();
    public void setRadius(float topLeft, float topRight, float bottomRight, fl...
        radii[0] = radii[1] = AndroidUtilities.dp(topLeft);
        radii[2] = radii[3] = AndroidUtilities.dp(topRight);
        radii[4] = radii[5] = AndroidUtilities.dp(bottomRight);
        radii[6] = radii[7] = AndroidUtilities.dp(bottomLeft);
        invalidatePath = true;
        invalidateSelf();
    @Override
    protected void onBoundsChange(Rect bounds) {
        invalidatePath = true;
    @Override
    public void draw(Canvas canvas) {
        if (invalidatePath) {
            invalidatePath = false;
            path.reset();
            AndroidUtilities.rectTmp.set(getBounds());
            path.addRoundRect(AndroidUtilities.rectTmp, radii, Path.Direction....
        canvas.drawPath(path, maskPaint);
    @Override
    public void setAlpha(int alpha) {
    @Override
```

```
    public void setColorFilter(ColorFilter colorFilter) {
    @Override
    public int getOpacity() {
        return PixelFormat.UNKNOWN;
public static void setColorFilter(Context context, ImageView imageView, int co...
    imageView.setColorFilter(new PorterDuffColorFilter(ContextCompat.getColor(...
public static void setColorFilter(ImageView imageView, int color) {
    imageView.setColorFilter(new PorterDuffColorFilter(color, PorterDuff.Mode....
private static Method StateListDrawable_getStateDrawableMethod;
@SuppressLint("PrivateApi")
private static Drawable getStateDrawable(Drawable drawable, int index) {
    if (Build.VERSION.SDK_INT >= 29 && drawable instanceof StateListDrawable) {
        return ((StateListDrawable) drawable).getStateDrawable(index);
    } else {
        if (StateListDrawable_getStateDrawableMethod == null) {
            try {
                StateListDrawable_getStateDrawableMethod = StateListDrawable.c...
            } catch (Throwable ignore) {
        if (StateListDrawable_getStateDrawableMethod == null) {
            return null;
        try {
            return (Drawable) StateListDrawable_getStateDrawableMethod.invoke(...
        } catch (Exception ignore) {
        return null;
public static void setSelectorDrawableColor(Drawable drawable, int color, bool...
    if (drawable instanceof StateListDrawable) {
        try {
            Drawable state;
            if (selected) {
                state = getStateDrawable(drawable, 0);
                if (state instanceof ShapeDrawable) {
                    ((ShapeDrawable) state).getPaint().setColor(color);
                } else {
                    state.setColorFilter(new PorterDuffColorFilter(color, Port...
                state = getStateDrawable(drawable, 1);
            } else {
                state = getStateDrawable(drawable, 2);
            if (state instanceof ShapeDrawable) {
                ((ShapeDrawable) state).getPaint().setColor(color);
            } else {
                state.setColorFilter(new PorterDuffColorFilter(color, PorterDu...
        } catch (Throwable ignore) {
    } else if (Build.VERSION.SDK_INT >= 21 && drawable instanceof RippleDrawab...
        RippleDrawable rippleDrawable = (RippleDrawable) drawable;
        if (selected) {
            rippleDrawable.setColor(new ColorStateList(
                    new int[][]{StateSet.WILD_CARD},
                    new int[]{color}
            ));
        } else {
```

```
        if (rippleDrawable.getNumberOfLayers() > 0) {
            Drawable drawable1 = rippleDrawable.getDrawable(0);
            if (drawable1 instanceof ShapeDrawable) {
                ((ShapeDrawable) drawable1).getPaint().setColor(color);
            } else {
                drawable1.setColorFilter(new PorterDuffColorFilter(color, ...
public static void setPressedBackground(View imageView) {
    if (Build.VERSION.SDK_INT >= 21) {
        imageView.setBackground(createSelectorDrawable(getPressedColor()));
public static Drawable createSelectorDrawable(int color) {
    return createSelectorDrawable(color, 1, -1);
public static Drawable createSelectorDrawable(int color, int maskType) {
    return createSelectorDrawable(color, maskType, -1);
public static Drawable createSelectorDrawable(int color, int maskType, int rad...
    Drawable drawable;
    if (Build.VERSION.SDK_INT >= 21) {
        Drawable maskDrawable = null;
        if ((maskType == 1 || maskType == 5) && Build.VERSION.SDK_INT >= 23) {
            maskDrawable = null;
        } else if (maskType == 1 || maskType == 3 || maskType == 4 || maskType...
            maskPaint.setColor(0xffffffff);
            maskDrawable = new Drawable() {
                RectF rect;
                @Override
                public void draw(Canvas canvas) {
                    android.graphics.Rect bounds = getBounds();
                    if (maskType == 7) {
                        if (rect == null) {
                            rect = new RectF();
                        rect.set(bounds);
                        canvas.drawRoundRect(rect, AndroidUtilities.dp(6), And...
                    } else {
                        int rad;
                        if (maskType == 1 || maskType == 6) {
                            rad = AndroidUtilities.dp(20);
                        } else if (maskType == 3) {
                            rad = (Math.max(bounds.width(), bounds.height()) /...
                        } else {
                            rad = (int) Math.ceil(Math.sqrt((bounds.left - bou...
                        canvas.drawCircle(bounds.centerX(), bounds.centerY(), ...
                @Override
                public void setAlpha(int alpha) {
                @Override
                public void setColorFilter(ColorFilter colorFilter) {
                @Override
                public int getOpacity() {
                    return PixelFormat.UNKNOWN;
        } else if (maskType == 2) {
            maskDrawable = new ColorDrawable(0xffffffff);
        ColorStateList colorStateList = new ColorStateList(
```

```
            new int[][]{StateSet.WILD_CARD},
            new int[]{color}
        );
        RippleDrawable rippleDrawable = new RippleDrawable(colorStateList, nul...
        if (Build.VERSION.SDK_INT >= 23) {
            if (maskType == 1) {
                rippleDrawable.setRadius(radius <= 0 ? AndroidUtilities.dp(20)...
            } else if (maskType == 5) {
                rippleDrawable.setRadius(RippleDrawable.RADIUS_AUTO);
        return rippleDrawable;
    } else {
        StateListDrawable stateListDrawable = new StateListDrawable();
        stateListDrawable.addState(new int[]{android.R.attr.state_pressed}, ne...
        stateListDrawable.addState(new int[]{android.R.attr.state_selected}, n...
        stateListDrawable.addState(StateSet.WILD_CARD, new ColorDrawable(0x000...
        return stateListDrawable;
@TargetApi(21)
@SuppressLint("DiscouragedPrivateApi")
public static void setRippleDrawableForceSoftware(RippleDrawable drawable) {
    if (drawable == null) {
        return;
    try {
        Method method = RippleDrawable.class.getDeclaredMethod("setForceSoftwa...
        method.invoke(drawable, true);
    } catch (Throwable ignore) {
public static Drawable createRoundRectDrawable(int topRad, int bottomRad, int ...
    ShapeDrawable defaultDrawable = new ShapeDrawable(new RoundRectShape(new f...
    defaultDrawable.getPaint().setColor(defaultColor);
    return defaultDrawable;
public static Drawable createRoundRectDrawable(int rad, int defaultColor) {
    ShapeDrawable defaultDrawable = new ShapeDrawable(new RoundRectShape(new f...
    defaultDrawable.getPaint().setColor(defaultColor);
    return defaultDrawable;
public static Drawable createEmojiIconSelectorDrawable(Context context, int re...
    Resources resources = context.getResources();
    Drawable defaultDrawable = resources.getDrawable(resource).mutate();
    if (defaultColor != 0) {
        defaultDrawable.setColorFilter(new PorterDuffColorFilter(defaultColor,...
    Drawable pressedDrawable = resources.getDrawable(resource).mutate();
    if (pressedColor != 0) {
        pressedDrawable.setColorFilter(new PorterDuffColorFilter(pressedColor,...
    StateListDrawable stateListDrawable = new StateListDrawable() {
        @Override
        public boolean selectDrawable(int index) {
            if (Build.VERSION.SDK_INT < 21) {
                Drawable drawable = Theme.getStateDrawable(this, index);
                ColorFilter colorFilter = null;
                if (drawable instanceof BitmapDrawable) {
                    colorFilter = ((BitmapDrawable) drawable).getPaint().getCo...
                } else if (drawable instanceof NinePatchDrawable) {
```

```
                colorFilter = ((NinePatchDrawable) drawable).getPaint().ge...
            boolean result = super.selectDrawable(index);
            if (colorFilter != null) {
                drawable.setColorFilter(colorFilter);
            return result;
        return super.selectDrawable(index);
    stateListDrawable.setEnterFadeDuration(1);
    stateListDrawable.setExitFadeDuration(200);
    stateListDrawable.addState(new int[]{android.R.attr.state_selected}, press...
    stateListDrawable.addState(new int[]{}, defaultDrawable);
    return stateListDrawable;
public static Bitmap createBitmap(int width, int height, int color) {
    Bitmap bitmap = Bitmap.createBitmap(width, height,
        Bitmap.Config.ARGB_8888);
    bitmap.eraseColor(color);
    Paint paint = new Paint();
    paint.setTextSize(100);
    paint.setColor(Color.GREEN);
    paint.setFlags(Paint.ANTI_ALIAS_FLAG);
    paint.setStyle(Paint.Style.FILL);
    return bitmap;
public static Bitmap drawBitmapBg(int color, Bitmap orginBitmap) {
    Paint paint = new Paint();
    paint.setColor(color);
    Bitmap bitmap = Bitmap.createBitmap(orginBitmap.getWidth(),
        orginBitmap.getHeight(), orginBitmap.getConfig());
    Canvas canvas = new Canvas(orginBitmap);
    canvas.drawRect(0, 0, orginBitmap.getWidth(), orginBitmap.getHeight(), pai...
    canvas.drawBitmap(orginBitmap, 0, 0, paint);
    return bitmap;
public static Drawable createSimpleSelectorRoundRectDrawable(int rad, int defa...
    return createSimpleSelectorRoundRectDrawable(rad, defaultColor, pressedCol...
public static Drawable createSimpleSelectorRoundRectDrawable(int rad, int defa...
    ShapeDrawable defaultDrawable = new ShapeDrawable(new RoundRectShape(new f...
    defaultDrawable.getPaint().setColor(defaultColor);
    ShapeDrawable pressedDrawable = new ShapeDrawable(new RoundRectShape(new f...
    pressedDrawable.getPaint().setColor(maskColor);
    if (Build.VERSION.SDK_INT >= 21) {
        ColorStateList colorStateList = new ColorStateList(
            new int[][]{StateSet.WILD_CARD},
            new int[]{pressedColor}
        );
        return new RippleDrawable(colorStateList, defaultDrawable, pressedDraw...
    } else {
        StateListDrawable stateListDrawable = new StateListDrawable();
        stateListDrawable.addState(new int[]{android.R.attr.state_pressed}, pr...
        stateListDrawable.addState(new int[]{android.R.attr.state_selected}, p...
        stateListDrawable.addState(StateSet.WILD_CARD, defaultDrawable);
        return stateListDrawable;
public static Drawable getRoundRectSelectorDrawable(int color) {
```

```
        return getRoundRectSelectorDrawable(AndroidUtilities.dp(3), color);
    public static Drawable getRoundRectSelectorDrawable(int corners, int color) {
        if (Build.VERSION.SDK_INT >= 21) {
            Drawable maskDrawable = createRoundRectDrawable(corners, 0xffffffff);
            ColorStateList colorStateList = new ColorStateList(
                    new int[][]{StateSet.WILD_CARD},
                    new int[]{(color & 0x00ffffff) | 0x19000000}
            );
            return new RippleDrawable(colorStateList, null, maskDrawable);
        } else {
            StateListDrawable stateListDrawable = new StateListDrawable();
            stateListDrawable.addState(new int[]{android.R.attr.state_pressed}, cr...
            stateListDrawable.addState(new int[]{android.R.attr.state_selected}, c...
            stateListDrawable.addState(StateSet.WILD_CARD, new ColorDrawable(0x000...
            return stateListDrawable;
    public static void setDrawableColor(Drawable drawable, int color) {
        if (drawable == null) {
            return;
        if (drawable instanceof ShapeDrawable) {
            ((ShapeDrawable) drawable).getPaint().setColor(color);
        } else {
            drawable.setColorFilter(new PorterDuffColorFilter(color, PorterDuff.Mo...
    public static int multAlpha(int color, float multiply) {
        if (multiply == 1f)
            return color;
        return ColorUtils.setAlphaComponent(color, MathUtils.clamp((int) (Color.al...
public class RadioCell extends FrameLayout {
    private TextView textView;
    private RadioButton radioButton;
    private boolean needDivider;
    private boolean isRTL = false;
    public RadioCell(Context context, boolean isRTL) {
        this(context, false, 15, isRTL);
    public RadioCell(Context context, boolean dialog, int padding, boolean isRTL) {
        super(context);
        this.isRTL = isRTL;
        textView = new TextView(context);
        if (dialog) {
            textView.setTextColor(ContextCompat.getColor(context, R.color.colorDar...
        } else {
            textView.setTextColor(ContextCompat.getColor(context, R.color.colorDar...
        textView.setTextSize(TypedValue.COMPLEX_UNIT_DIP, 16);
        textView.setLines(1);
        textView.setMaxLines(1);
        textView.setSingleLine(true);
        textView.setEllipsize(TextUtils.TruncateAt.END);
        textView.setGravity((isRTL ? Gravity.END : Gravity.START) | Gravity.CENTER...
        addView(textView, LayoutHelper.createFrame(LayoutHelper.MATCH_PARENT, Layo...
        radioButton = new RadioButton(context);
        radioButton.setSize(AndroidUtilities.dp(20));
```

```
        if (dialog) {
            radioButton.setColor(ContextCompat.getColor(context, R.color.color999)...
        } else {
            radioButton.setColor(ContextCompat.getColor(context, R.color.color999)...
        addView(radioButton, LayoutHelper.createFrame(22, 22, (isRTL ? Gravity.STA...
        setBackground(ContextCompat.getDrawable(context, R.drawable.layout_bg));
    @Override
    protected void onMeasure(int widthMeasureSpec, int heightMeasureSpec) {
        setMeasuredDimension(MeasureSpec.getSize(widthMeasureSpec), AndroidUtiliti...
        int availableWidth = getMeasuredWidth() - getPaddingLeft() - getPaddingRig...
        radioButton.measure(MeasureSpec.makeMeasureSpec(AndroidUtilities.dp(22), M...
        textView.measure(MeasureSpec.makeMeasureSpec(availableWidth, MeasureSpec.E...
    public void setTextColor(int color) {
        textView.setTextColor(color);
    public void setText(String text, boolean checked, boolean divider) {
        textView.setText(text);
        radioButton.setChecked(checked, false);
        needDivider = divider;
        setWillNotDraw(!divider);
    public boolean isChecked() {
        return radioButton.isChecked();
    public void setChecked(boolean checked, boolean animated) {
        radioButton.setChecked(checked, animated);
    public void setEnabled(boolean value, ArrayList<Animator> animators) {
        super.setEnabled(value);
        if (animators != null) {
            animators.add(ObjectAnimator.ofFloat(textView, View.ALPHA, value ? 1.0...
            animators.add(ObjectAnimator.ofFloat(radioButton, View.ALPHA, value ? ...
        } else {
            textView.setAlpha(value ? 1.0f : 0.5f);
            radioButton.setAlpha(value ? 1.0f : 0.5f);
    public void hideRadioButton() {
        radioButton.setVisibility(View.GONE);
    @Override
    protected void onDraw(Canvas canvas) {
        if (needDivider) {
            Paint dividerPaint = new Paint();
            dividerPaint.setStrokeWidth(1);
            dividerPaint.setColor(ContextCompat.getColor(getContext(), R.color.col...
            canvas.drawLine(isRTL ? 0 : AndroidUtilities.dp(20), getMeasuredHeight...
    @Override
    public void onInitializeAccessibilityNodeInfo(AccessibilityNodeInfo info) {
        super.onInitializeAccessibilityNodeInfo(info);
        info.setClassName("android.widget.RadioButton");
        info.setCheckable(true);
        info.setChecked(isChecked());
public class FilterImageView extends ShapeableImageView {
    public final float[] BG_PRESSED = new float[]{1, 0, 0, 0, -50, 0, 1,
            0, 0, -50, 0, 0, 1, 0, -50, 0, 0, 0, 1, 0};
    public final float[] BG_NOT_PRESSED = new float[]{1, 0, 0, 0, 0, 0,
```

```
        1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0};
    public FilterImageView(Context context) {
        super(context);
        init();
    public FilterImageView(Context context, AttributeSet attrs) {
        super(context, attrs);
        init();
    public FilterImageView(Context context, AttributeSet attrs, int defStyle) {
        super(context, attrs, defStyle);
        init();
    void init() {
        setStrokeColorResource(R.color.borderColor);
        setStrokeWidth(AndroidUtilities.dp(0.1f));
        setShapeAppearanceModel(getShapeAppearanceModel()
                .toBuilder()
                .setAllCorners(CornerFamily.ROUNDED, AndroidUtilities.dp(10))
                .build());
    public void setStrokeWidth(float width) {
        super.setStrokeWidth(AndroidUtilities.dp(width));
    public void setAllCorners(int cornerSize) {
        setShapeAppearanceModel(getShapeAppearanceModel()
                .toBuilder()
                .setAllCorners(CornerFamily.ROUNDED, AndroidUtilities.dp(cornerSize))
                .build());
    public void setCorners(int topLeft, int topRight, int bottomLeft, int bottomRi...
        setShapeAppearanceModel(getShapeAppearanceModel()
                .toBuilder()
                .setTopLeftCorner(CornerFamily.ROUNDED, AndroidUtilities.dp(topLeft))
                .setTopRightCorner(CornerFamily.ROUNDED, AndroidUtilities.dp(topRi...
                .setBottomRightCorner(CornerFamily.ROUNDED, AndroidUtilities.dp(bo...
                .setBottomLeftCorner(CornerFamily.ROUNDED, AndroidUtilities.dp(bot...
                .build());
    @Override
    public void setPressed(boolean pressed) {
        updateView(pressed);
        super.setPressed(pressed);
    private void updateView(boolean pressed) {
        if (pressed) {
            this.setDrawingCacheEnabled(true);
            this.setColorFilter(new ColorMatrixColorFilter(BG_PRESSED));
            this.getDrawable().setColorFilter(new ColorMatrixColorFilter(BG_PRESSE...
        } else {
            this.setColorFilter(new ColorMatrixColorFilter(BG_NOT_PRESSED));
            this.getDrawable().setColorFilter(
                    new ColorMatrixColorFilter(BG_NOT_PRESSED));
public class ReactionsContainerLayout extends FrameLayout {
    public final static Property<ReactionsContainerLayout, Float> TRANSITION_PROGR...
        @Override
        public Float get(ReactionsContainerLayout reactionsContainerLayout) {
            return reactionsContainerLayout.transitionProgress;
```

```
    @Override
    public void set(ReactionsContainerLayout object, Float value) {
        object.setTransitionProgress(value);
private final static int ALPHA_DURATION = 150;
private final static float SIDE_SCALE = 0.6f;
private final static float SCALE_PROGRESS = 0.75f;
private final static float CLIP_PROGRESS = 0.25f;
public final RecyclerListView recyclerListView;
private final Paint bgPaint = new Paint(Paint.ANTI_ALIAS_FLAG);
private final Paint leftShadowPaint = new Paint(Paint.ANTI_ALIAS_FLAG);
private final Paint rightShadowPaint = new Paint(Paint.ANTI_ALIAS_FLAG);
private float leftAlpha, rightAlpha;
private float transitionProgress = 1f;
private final RectF rect = new RectF();
private final Path mPath = new Path();
private float radius = AndroidUtilities.dp(72);
private final float bigCircleRadius = AndroidUtilities.dp(8);
private final float smallCircleRadius = bigCircleRadius / 2;
private final int bigCircleOffset = AndroidUtilities.dp(36);
ValueAnimator cancelPressedAnimation;
private List<ReactionSticker> reactionsList = new ArrayList<>();
private final LinearLayoutManager linearLayoutManager;
private final RecyclerView.Adapter listAdapter;
private final int[] location = new int[2];
private ReactionsContainerDelegate delegate;
private final Rect shadowPad = new Rect();
private final Drawable shadow;
private final boolean animationEnabled;
private String pressedReaction;
private int pressedReactionPosition;
private float pressedProgress;
private float cancelPressedProgress;
private float pressedViewScale;
private float otherViewsScale;
private boolean clicked;
long lastReactionSentTime;
public ReactionsContainerLayout(@NonNull Context context) {
    super(context);
    Object object = EndpointManager.getInstance().invoke("reaction_sticker", n...
    List<ReactionSticker> list = (List<ReactionSticker>) object;
    if (list == null) {
        list = new ArrayList<>();
    reactionsList.addAll(list);
    animationEnabled = true;
    shadow = ContextCompat.getDrawable(context, R.mipmap.reactions_bubble_shad...
    shadowPad.left = shadowPad.top = shadowPad.right = shadowPad.bottom = Andr...
    shadow.setColorFilter(new PorterDuffColorFilter(ContextCompat.getColor(get...
    recyclerListView = new RecyclerListView(context) {
        @Override
        public boolean drawChild(Canvas canvas, View child, long drawingTime) {
```

```
        if (((ReactionHolderView) child).currentReaction.name.equals(press...
            return true;
        return super.drawChild(canvas, child, drawingTime);
    linearLayoutManager = new LinearLayoutManager(context, LinearLayoutManager...
    recyclerListView.addItemDecoration(new RecyclerView.ItemDecoration() {
        @Override
        public void getItemOffsets(@NonNull Rect outRect, @NonNull View view, ...
            super.getItemOffsets(outRect, view, parent, state);
            int position = parent.getChildAdapterPosition(view);
            if (position == 0) {
                outRect.left = AndroidUtilities.dp(6);
            outRect.right = AndroidUtilities.dp(4);
            if (position == listAdapter.getItemCount() - 1) {
                outRect.right = AndroidUtilities.dp(6);
    });
    recyclerListView.setLayoutManager(linearLayoutManager);
    recyclerListView.setOverScrollMode(View.OVER_SCROLL_NEVER);
    recyclerListView.setAdapter(listAdapter = new RecyclerView.Adapter() {
        @NonNull
        @Override
        public RecyclerView.ViewHolder onCreateViewHolder(@NonNull ViewGroup p...
            ReactionHolderView hv = new ReactionHolderView(context);
            int size = getLayoutParams().height - getPaddingTop() - getPadding...
            hv.setLayoutParams(new RecyclerView.LayoutParams(size - AndroidUti...
            return new RecyclerListView.Holder(hv);
        @Override
        public void onBindViewHolder(@NonNull RecyclerView.ViewHolder holder, ...
            ReactionHolderView h = (ReactionHolderView) holder.itemView;
            h.setScaleX(1);
            h.setScaleY(1);
            h.setReaction(reactionsList.get(position));
        @Override
        public int getItemCount() {
            return reactionsList.size();
    });
    recyclerListView.addOnScrollListener(new LeftRightShadowsListener());
    recyclerListView.addOnScrollListener(new RecyclerView.OnScrollListener() {
        @Override
        public void onScrolled(@NonNull RecyclerView recyclerView, int dx, int...
            if (recyclerView.getChildCount() > 2) {
                float sideDiff = 1f - SIDE_SCALE;
                recyclerView.getLocationInWindow(location);
                int rX = location[0];
                View ch1 = recyclerView.getChildAt(0);
                ch1.getLocationInWindow(location);
                int ch1X = location[0];
                int dX1 = ch1X - rX;
                float s1 = SIDE_SCALE + (1f - Math.min(1, -Math.min(dX1, 0f) /...
                if (Float.isNaN(s1)) s1 = 1f;
                ((ReactionHolderView) ch1).sideScale = s1;
```

```
            View ch2 = recyclerView.getChildAt(recyclerView.getChildCount(...
            ch2.getLocationInWindow(location);
            int ch2X = location[0];
            int dX2 = rX + recyclerView.getWidth() - (ch2X + ch2.getWidth());
            float s2 = SIDE_SCALE + (1f - Math.min(1, -Math.min(dX2, 0f) /...
            if (Float.isNaN(s2)) s2 = 1f;
            ((ReactionHolderView) ch2).sideScale = s2;
        for (int i = 1; i < recyclerListView.getChildCount() - 1; i++) {
            View ch = recyclerListView.getChildAt(i);
            ((ReactionHolderView) ch).sideScale = 1f;
        invalidate();
    });
    recyclerListView.addItemDecoration(new RecyclerView.ItemDecoration() {
        @Override
        public void getItemOffsets(@NonNull Rect outRect, @NonNull View view, ...
            int i = parent.getChildAdapterPosition(view);
            if (i == 0)
                outRect.left = AndroidUtilities.dp(8);
            if (i == listAdapter.getItemCount() - 1)
                outRect.right = AndroidUtilities.dp(8);
    });
    addView(recyclerListView, LayoutHelper.createFrame(LayoutHelper.MATCH_PARE...
    invalidateShaders();
    bgPaint.setColor(ContextCompat.getColor(getContext(), R.color.screen_bg));
    startEnterAnimation();
public void setDelegate(ReactionsContainerDelegate delegate) {
    this.delegate = delegate;
@SuppressLint("NotifyDataSetChanged")
private void setReactionsList(List<ReactionSticker> reactionsList) {
    this.reactionsList = reactionsList;
    int size = getLayoutParams().height - getPaddingTop() - getPaddingBottom();
    if (size * reactionsList.size() < AndroidUtilities.dp(200)) {
        getLayoutParams().width = ViewGroup.LayoutParams.WRAP_CONTENT;
    listAdapter.notifyDataSetChanged();
HashSet<View> lastVisibleViews = new HashSet<>();
HashSet<View> lastVisibleViewsTmp = new HashSet<>();
@Override
protected void dispatchDraw(Canvas canvas) {
    lastVisibleViewsTmp.clear();
    lastVisibleViewsTmp.addAll(lastVisibleViews);
    lastVisibleViews.clear();
    if (pressedReaction != null) {
        if (pressedProgress != 1f) {
            pressedProgress += 16f / 1500f;
            if (pressedProgress >= 1f) {
                pressedProgress = 1f;
            invalidate();
    float cPr = (Math.max(CLIP_PROGRESS, Math.min(transitionProgress, 1f)) - C...
    float br = bigCircleRadius * cPr, sr = smallCircleRadius * cPr;
    pressedViewScale = 1 + 2 * pressedProgress;
```

```
otherViewsScale = 1 - 0.15f * pressedProgress;
int s = canvas.save();
float pivotX = AndroidUtilities.isRTL ? getWidth() * 0.125f : getWidth() *...
if (transitionProgress <= SCALE_PROGRESS) {
   float sc = transitionProgress / SCALE_PROGRESS;
   canvas.scale(sc, sc, pivotX, getHeight() / 2f);
float lt = 0, rt = 1;
if (AndroidUtilities.isRTL) {
   rt = Math.max(CLIP_PROGRESS, transitionProgress);
} else {
   lt = (1f - Math.max(CLIP_PROGRESS, transitionProgress));
rect.set(getPaddingLeft() + (getWidth() - getPaddingRight()) * lt, getPadd...
radius = rect.height() / 2f;
shadow.setBounds((int) (getPaddingLeft() + (getWidth() - getPaddingRight()...
shadow.draw(canvas);
canvas.restoreToCount(s);
s = canvas.save();
if (transitionProgress <= SCALE_PROGRESS) {
   float sc = transitionProgress / SCALE_PROGRESS;
   canvas.scale(sc, sc, pivotX, getHeight() / 2f);
canvas.drawRoundRect(rect, radius, radius, bgPaint);
canvas.restoreToCount(s);
mPath.rewind();
mPath.addRoundRect(rect, radius, radius, Path.Direction.CW);
s = canvas.save();
if (transitionProgress <= SCALE_PROGRESS) {
   float sc = transitionProgress / SCALE_PROGRESS;
   canvas.scale(sc, sc, pivotX, getHeight() / 2f);
if (transitionProgress != 0 && getAlpha() == 1f) {
   int delay = 0;
   for (int i = 0; i < recyclerListView.getChildCount(); i++) {
      ReactionHolderView view = (ReactionHolderView) recyclerListView.ge...
      checkPressedProgress(canvas, view);
      if (view.getX() + view.getMeasuredWidth() / 2f > 0 && view.getX() ...
         if (!lastVisibleViewsTmp.contains(view)) {
            view.play(delay);
            delay += 30;
         lastVisibleViews.add(view);
      } else if (!view.isEnter) {
         view.resetAnimation();
canvas.clipPath(mPath);
canvas.translate((AndroidUtilities.isRTL ? -1 : 1) * getWidth() * (1f - tr...
super.dispatchDraw(canvas);
float p = clamp(leftAlpha * transitionProgress, 1f, 1f);
leftShadowPaint.setAlpha((int) (p * 0xFF));
canvas.drawRect(rect, leftShadowPaint);
p = clamp(rightAlpha * transitionProgress, 1f, 1f);
rightShadowPaint.setAlpha((int) (p * 0xFF));
canvas.drawRect(rect, rightShadowPaint);
canvas.restoreToCount(s);
```

```
    canvas.save();
    canvas.clipRect(0, rect.bottom, getMeasuredWidth(), getMeasuredHeight());
    float cx = AndroidUtilities.isRTL ? bigCircleOffset : getWidth() - bigCirc...
    int sPad = AndroidUtilities.dp(3);
    shadow.setBounds((int) (cx - br - sPad * cPr), (int) (cy - br - sPad * cPr...
    shadow.draw(canvas);
    canvas.drawCircle(cx, cy, br, bgPaint);
    cx = AndroidUtilities.isRTL ? bigCircleOffset - bigCircleRadius : getWidth...
    cy = getHeight() - smallCircleRadius - sPad;
    sPad = -AndroidUtilities.dp(1);
    shadow.setBounds((int) (cx - br - sPad * cPr), (int) (cy - br - sPad * cPr...
    shadow.draw(canvas);
    canvas.drawCircle(cx, cy, sr, bgPaint);
    canvas.restore();
  public int getTotalWidth() {
    return AndroidUtilities.dp(36) * reactionsList.size() + AndroidUtilities.d...
  public int getItemsCount() {
    return reactionsList.size();
  public float clamp(float value, float top, float bottom) {
    return Math.max(Math.min(value, top), bottom);
  private void checkPressedProgress(Canvas canvas, ReactionHolderView view) {
    if (view.currentReaction.name.equals(pressedReaction)) {
      view.setPivotX(view.getMeasuredWidth() >> 1);
      view.setScaleX(pressedViewScale);
      view.setScaleY(pressedViewScale);
      if (!clicked) {
        if (cancelPressedAnimation == null) {
          view.pressedBackupImageView.setVisibility(View.VISIBLE);
          view.pressedBackupImageView.setAlpha(1f);
        } else {
          view.pressedBackupImageView.setAlpha(1f - cancelPressedProgress);
        if (pressedProgress == 1f) {
          clicked = true;
          if (System.currentTimeMillis() - lastReactionSentTime > 300) {
            lastReactionSentTime = System.currentTimeMillis();
            int[] location = new int[2];
            view.getLocationOnScreen(location);
            delegate.onReactionClicked(view, view.currentReaction.name...
      canvas.save();
      float x = recyclerListView.getX() + view.getX();
      float additionalWidth = (view.getMeasuredWidth() * view.getScaleX() - ...
      if (x - additionalWidth < 0 && view.getTranslationX() >= 0) {
        view.setTranslationX(-(x - additionalWidth));
      } else if (x + view.getMeasuredWidth() + additionalWidth > getMeasured...
        view.setTranslationX(getMeasuredWidth() - x - view.getMeasuredWidt...
      } else {
        view.setTranslationX(0);
      x = recyclerListView.getX() + view.getX();
      canvas.translate(x, recyclerListView.getY() + view.getY());
      canvas.scale(view.getScaleX(), view.getScaleY(), view.getPivotX(), vie...
```

```
            view.draw(canvas);
            canvas.restore();
        } else {
            int position = recyclerListView.getChildAdapterPosition(view);
            float translationX;
            translationX = (view.getMeasuredWidth() * (pressedViewScale - 1f)) / 3...
            if (position < pressedReactionPosition) {
                view.setPivotX(0);
                view.setTranslationX(-translationX);
            } else {
                view.setPivotX(view.getMeasuredWidth());
                view.setTranslationX(translationX);
            view.setPivotY(view.pressedBackupImageView.getY() + view.pressedBackup...
            view.setScaleX(otherViewsScale);
            view.setScaleY(otherViewsScale);
            view.pressedBackupImageView.setVisibility(View.VISIBLE);
    @Override
    protected void onSizeChanged(int w, int h, int oldw, int oldh) {
        super.onSizeChanged(w, h, oldw, oldh);
        invalidateShaders();
    private void invalidateShaders() {
        int dp = AndroidUtilities.dp(30);
        float cy = getHeight() / 2f;
        int clr = ContextCompat.getColor(getContext(), R.color.screen_bg);
        leftShadowPaint.setShader(new LinearGradient(0, cy, dp, cy, clr, Color.TRA...
        rightShadowPaint.setShader(new LinearGradient(getWidth(), cy, getWidth() -...
        invalidate();
    public void setTransitionProgress(float transitionProgress) {
        this.transitionProgress = transitionProgress;
        invalidate();
    public void startEnterAnimation() {
        setTransitionProgress(0);
        setAlpha(1f);
        ObjectAnimator animator = ObjectAnimator.ofFloat(this, ReactionsContainerL...
        animator.setInterpolator(new OvershootInterpolator(1.004f));
        animator.start();
    private final class LeftRightShadowsListener extends RecyclerView.OnScrollList...
        private boolean leftVisible, rightVisible;
        private ValueAnimator leftAnimator, rightAnimator;
        @Override
        public void onScrolled(@NonNull RecyclerView recyclerView, int dx, int dy) {
            boolean l = linearLayoutManager.findFirstVisibleItemPosition() != 0;
            if (l != leftVisible) {
                if (leftAnimator != null)
                    leftAnimator.cancel();
                leftAnimator = startAnimator(leftAlpha, l ? 1 : 0, aFloat -> {
                    leftShadowPaint.setAlpha((int) ((leftAlpha = aFloat) * 0xFF));
                    invalidate();
                }, () -> leftAnimator = null);
                leftVisible = l;
```

```
        boolean r = linearLayoutManager.findLastVisibleItemPosition() != listA...
        if (r != rightVisible) {
            if (rightAnimator != null)
                rightAnimator.cancel();
            rightAnimator = startAnimator(rightAlpha, r ? 1 : 0, aFloat -> {
                rightShadowPaint.setAlpha((int) ((rightAlpha = aFloat) * 0xFF));
                invalidate();
            }, () -> rightAnimator = null);
            rightVisible = r;
    private ValueAnimator startAnimator(float fromAlpha, float toAlpha, Consum...
        ValueAnimator a = ValueAnimator.ofFloat(fromAlpha, toAlpha).setDuratio...
        a.addUpdateListener(animation -> callback.accept((Float) animation.get...
        a.addListener(new AnimatorListenerAdapter() {
            @Override
            public void onAnimationEnd(Animator animation) {
                onEnd.run();
        });
        a.start();
        return a;
public final class ReactionHolderView extends FrameLayout {
    public RLottieImageView pressedBackupImageView;
    public ReactionSticker currentReaction;
    public float sideScale = 1f;
    private boolean isEnter;
    Runnable playRunnable = new Runnable() {
        @Override
        public void run() {
            if (pressedBackupImageView.getAnimatedDrawable() != null && !press...
                pressedBackupImageView.getAnimatedDrawable().start();
    ReactionHolderView(Context context) {
        super(context);
        pressedBackupImageView = new RLottieImageView(context) {
            @Override
            public void invalidate() {
                super.invalidate();
                ReactionsContainerLayout.this.invalidate();
        addView(pressedBackupImageView, LayoutHelper.createFrame(34, 34, Gravi...
    private void setReaction(ReactionSticker react) {
        currentReaction = react;
        RLottieDrawable drawable = new RLottieDrawable(getContext(), currentRe...
        pressedBackupImageView.setAutoRepeat(false);
        pressedBackupImageView.setAnimation(drawable);
        pressedBackupImageView.playAnimation();
        isEnter = false;
    @Override
    protected void onAttachedToWindow() {
        super.onAttachedToWindow();
    public boolean play(int delay) {
        if (!animationEnabled) {
            resetAnimation();
```

```
        isEnter = true;
        return false;
    AndroidUtilities.cancelRunOnUIThread(playRunnable);
    if (pressedBackupImageView.getAnimatedDrawable() != null) {
        pressedBackupImageView.getAnimatedDrawable().setCurrentFrame(0, fa...
        if (delay > 0) {
            AndroidUtilities.runOnUIThread(playRunnable, delay);
        } else {
            playRunnable.run();
    isEnter = true;
    return true;
public void resetAnimation() {
    AndroidUtilities.cancelRunOnUIThread(playRunnable);
    if (pressedBackupImageView.getAnimatedDrawable() != null) {
        if (animationEnabled) {
            pressedBackupImageView.getAnimatedDrawable().setCurrentFrame(0...
        } else {
            pressedBackupImageView.getAnimatedDrawable().setCurrentFrame(p...
    isEnter = false;
Runnable longPressRunnable = new Runnable() {
    @Override
    public void run() {
        performHapticFeedback(HapticFeedbackConstants.LONG_PRESS);
        pressedReactionPosition = reactionsList.indexOf(currentReaction);
        pressedReaction = currentReaction.name;
        ReactionsContainerLayout.this.invalidate();
float pressedX, pressedY;
boolean pressed;
@Override
public boolean onTouchEvent(MotionEvent event) {
    if (cancelPressedAnimation != null) {
        return false;
    if (event.getAction() == MotionEvent.ACTION_DOWN) {
        pressed = true;
        pressedX = event.getX();
        pressedY = event.getY();
        if (sideScale == 1f) {
    float touchSlop = ViewConfiguration.get(getContext()).getScaledTouchSl...
    boolean cancelByMove = event.getAction() == MotionEvent.ACTION_MOVE &&...
    if (cancelByMove || event.getAction() == MotionEvent.ACTION_UP || even...
        if (event.getAction() == MotionEvent.ACTION_UP && pressed && (pres...
            clicked = true;
            if (System.currentTimeMillis() - lastReactionSentTime > 300) {
                lastReactionSentTime = System.currentTimeMillis();
                int[] location = new int[2];
                this.getLocationOnScreen(location);
                delegate.onReactionClicked(this, currentReaction.name, pre...
        if (!clicked) {
            cancelPressed();
        pressed = false;
```

```
        return true;
    private void cancelPressed() {
        if (pressedReaction != null) {
            cancelPressedProgress = 0f;
            float fromProgress = pressedProgress;
            cancelPressedAnimation = ValueAnimator.ofFloat(0, 1f);
            cancelPressedAnimation.addUpdateListener(new ValueAnimator.AnimatorUpd...
                @Override
                public void onAnimationUpdate(ValueAnimator valueAnimator) {
                    cancelPressedProgress = (float) valueAnimator.getAnimatedValue();
                    pressedProgress = fromProgress * (1f - cancelPressedProgress);
                    ReactionsContainerLayout.this.invalidate();
            });
            cancelPressedAnimation.addListener(new AnimatorListenerAdapter() {
                @Override
                public void onAnimationEnd(Animator animation) {
                    super.onAnimationEnd(animation);
                    cancelPressedAnimation = null;
                    pressedProgress = 0;
                    pressedReaction = null;
                    ReactionsContainerLayout.this.invalidate();
            });
            cancelPressedAnimation.setDuration(150);
            cancelPressedAnimation.setInterpolator(CubicBezierInterpolator.DEFAULT);
            cancelPressedAnimation.start();
    public interface ReactionsContainerDelegate {
        void onReactionClicked(View v, String name, boolean longpress, int[] local...
    @Override
    protected void onAttachedToWindow() {
        super.onAttachedToWindow();
    @Override
    protected void onDetachedFromWindow() {
        super.onDetachedFromWindow();
    @Override
    public void setAlpha(float alpha) {
        if (getAlpha() != alpha && alpha == 0) {
            lastVisibleViews.clear();
            for (int i = 0; i < recyclerListView.getChildCount(); i++) {
                ReactionHolderView view = (ReactionHolderView) recyclerListView.ge...
                view.resetAnimation();
        super.setAlpha(alpha);
public class DrawableHelper {
    public static void setRoundBackground(View view, Drawable drawable) {
        if(Build.VERSION.SDK_INT >= Build.VERSION_CODES.JELLY_BEAN) {
            view.setBackground(drawable);
        } else {
            view.setBackgroundDrawable(drawable);
    public static Drawable getCornerDrawable(float topLeft,
                                             float topRight,
                                             float bottomLeft,
```

```
                                float bottomRight,
                                @ColorInt int color) {
        float[] outerR = new float[8];
        outerR[0] = topLeft;
        outerR[1] = topLeft;
        outerR[2] = topRight;
        outerR[3] = topRight;
        outerR[4] = bottomRight;
        outerR[5] = bottomRight;
        outerR[6] = bottomLeft;
        outerR[7] = bottomLeft;
        ShapeDrawable drawable = new ShapeDrawable();
        drawable.setShape(new RoundRectShape(outerR, null, null));
        drawable.getPaint().setColor(color);
        return drawable;
public class FadingTextViewLayout extends FrameLayout {
    private final ValueAnimator animator;
    private CharSequence text;
    private TextView foregroundView;
    private TextView currentView;
    private TextView nextView;
    public FadingTextViewLayout(Context context, AttributeSet attrs, int defStyle) {
        this(context,false);
    public FadingTextViewLayout(Context context) {
        this(context, false);
    public FadingTextViewLayout(Context context, boolean hasStaticChars) {
        super(context);
        for (int i = 0; i < 2 + (hasStaticChars ? 1 : 0); i++) {
            final TextView textView = new TextView(context);
            onTextViewCreated(textView);
            addView(textView);
            if (i == 0) {
                currentView = textView;
            } else {
                textView.setVisibility(GONE);
                if (i == 1) {
                    textView.setAlpha(0f);
                    nextView = textView;
                } else {
                    foregroundView = textView;
        animator = ValueAnimator.ofFloat(0f, 1f);
        animator.setDuration(200);
        animator.setInterpolator(null);
        animator.addUpdateListener(a -> {
            final float fraction = a.getAnimatedFraction();
            currentView.setAlpha(CubicBezierInterpolator.DEFAULT.getInterpolation(...
            nextView.setAlpha(CubicBezierInterpolator.DEFAULT.getInterpolation(1f ...
        });
        animator.addListener(new AnimatorListenerAdapter() {
            @Override
```

```
        public void onAnimationEnd(Animator animation) {
            currentView.setLayerType(LAYER_TYPE_NONE, null);
            nextView.setLayerType(LAYER_TYPE_NONE, null);
            nextView.setVisibility(GONE);
            if (foregroundView != null) {
                currentView.setText(text);
                foregroundView.setVisibility(GONE);
        @Override
        public void onAnimationStart(Animator animation) {
            currentView.setLayerType(LAYER_TYPE_HARDWARE, null);
            nextView.setLayerType(LAYER_TYPE_HARDWARE, null);
            if (ViewCompat.isAttachedToWindow(currentView)) {
                currentView.buildLayer();
            if (ViewCompat.isAttachedToWindow(nextView)) {
                nextView.buildLayer();
    });
    public void setText(CharSequence text) {
        setText(text, true, true);
    public void setText(CharSequence text, boolean animated) {
        setText(text, animated, true);
    public void setText(CharSequence text, boolean animated, boolean dontAnimateUn...
        if (!TextUtils.equals(text, currentView.getText())) {
            if (animator != null) {
                animator.end();
            this.text = text;
            if (animated) {
                if (dontAnimateUnchangedStaticChars && foregroundView != null) {
                    final int staticCharsCount = getStaticCharsCount();
                    if (staticCharsCount > 0) {
                        final CharSequence currentText = currentView.getText();
                        final int length = Math.min(staticCharsCount, Math.min(tex...
                        final List<Point> points = new ArrayList<>();
                        int startIndex = -1;
                        for (int i = 0; i < length; i++) {
                            final char c = currentText.charAt(i);
                            if (text.charAt(i) == c) {
                                if (startIndex >= 0) {
                                    points.add(new Point(startIndex, i));
                                    startIndex = -1;
                            } else if (startIndex == -1) {
                                startIndex = i;
                        if (startIndex == 0) {
                        } else if (startIndex > 0) {
                            points.add(new Point(startIndex, length));
                        } else {
                            points.add(new Point(length, 0));
                        if (!points.isEmpty()) {
                            final SpannableString foregroundText = new SpannableSt...
                            final SpannableString currentSpannableText = new Spann...
                            final SpannableString spannableText = new SpannableStr...
```

```
                int lastIndex = 0;
                for (int i = 0, N = points.size(); i < N; i++) {
                    final Point point = points.get(i);
                    if (point.y > point.x) {
                        foregroundText.setSpan(new ForegroundColorSpan...
                    if (point.x > lastIndex) {
                        currentSpannableText.setSpan(new ForegroundCol...
                        spannableText.setSpan(new ForegroundColorSpan(...
                    lastIndex = point.y;
                foregroundView.setVisibility(VISIBLE);
                foregroundView.setText(foregroundText);
                currentView.setText(currentSpannableText);
                text = spannableText;
            nextView.setVisibility(VISIBLE);
            nextView.setText(text);
            showNext();
        } else {
            currentView.setText(text);
    public CharSequence getText() {
        return text;
    public TextView getCurrentView() {
        return currentView;
    public TextView getNextView() {
        return nextView;
    private void showNext() {
        final TextView prevView = currentView;
        currentView = nextView;
        nextView = prevView;
        animator.start();
    protected void onTextViewCreated(TextView textView) {
        textView.setSingleLine(true);
        textView.setMaxLines(1);
    protected int getStaticCharsCount() {
        return 0;
public class AnimationProperties {
    public static OvershootInterpolator overshootInterpolator = new OvershootInter...
    public static abstract class FloatProperty<T> extends Property<T, Float> {
        public FloatProperty(String name) {
            super(Float.class, name);
        public abstract void setValue(T object, float value);
        @Override
        final public void set(T object, Float value) {
            setValue(object, value);
    public static abstract class IntProperty<T> extends Property<T, Integer> {
        public IntProperty(String name) {
            super(Integer.class, name);
        public abstract void setValue(T object, int value);
        @Override
        final public void set(T object, Integer value) {
            setValue(object, value);
```

```
    public static final Property<Paint, Integer> PAINT_ALPHA = new IntProperty<Pai...
        @Override
        public void setValue(Paint object, int value) {
            object.setAlpha(value);
        @Override
        public Integer get(Paint object) {
            return object.getAlpha();
    public static final Property<Paint, Integer> PAINT_COLOR = new IntProperty<Pai...
        @Override
        public void setValue(Paint object, int value) {
            object.setColor(value);
        @Override
        public Integer get(Paint object) {
            return object.getColor();
    public static final Property<ColorDrawable, Integer> COLOR_DRAWABLE_ALPHA = ne...
        @Override
        public void setValue(ColorDrawable object, int value) {
            object.setAlpha(value);
        @Override
        public Integer get(ColorDrawable object) {
            return object.getAlpha();
    public static final Property<ShapeDrawable, Integer> SHAPE_DRAWABLE_ALPHA = ne...
        @Override
        public void setValue(ShapeDrawable object, int value) {
            object.getPaint().setAlpha(value);
        @Override
        public Integer get(ShapeDrawable object) {
            return object.getPaint().getAlpha();
public class ContactEditText extends AppCompatAutoCompleteTextView {
    private int itemPadding;
    private int maxLength;
    public ContactEditText(Context context) {
        super(context);
        init();
    public ContactEditText(Context context, AttributeSet attrs) {
        super(context, attrs);
        init();
    public ContactEditText(Context context, AttributeSet attrs, int defStyleAttr) {
        super(context, attrs, defStyleAttr);
        init();
    private void init() {
        itemPadding = dip2px(getContext(), 3);
    public void setMaxLength(int maxLength) {
        this.maxLength = maxLength;
    @Override
    protected void onSelectionChanged(int selStart, int selEnd) {
        MyImageSpan[] spans = getText().getSpans(0, getText().length(), MyImageSpa...
        for (MyImageSpan myImageSpan : spans) {
            if (getText().getSpanEnd(myImageSpan) - 1 == selStart) {
                selStart = selStart + 1;
```

```
            setSelection(selStart);
            break;
    super.onSelectionChanged(selStart, selEnd);
  private void flushSpans() {
    Editable editText = getText();
    Spannable spannableString = new SpannableString(editText);
    MyImageSpan[] spans = spannableString.getSpans(0, editText.length(), MyIma...
    List<UnSpanText> texts = getAllTexts(spans, editText);
    for (UnSpanText unSpanText : texts) {
      if (!TextUtils.isEmpty(unSpanText.showText.toString().trim())) {
        generateOneSpan(spannableString, unSpanText);
    setText(spannableString);
    setSelection(spannableString.length());
  private List<UnSpanText> getAllTexts(MyImageSpan[] spans, Editable edittext) {
    List<UnSpanText> texts = new ArrayList<>();
    int start;
    int end;
    CharSequence text;
    List<Integer> sortStartEnds = new ArrayList<>();
    sortStartEnds.add(0);
    for (MyImageSpan myImageSpan : spans) {
      sortStartEnds.add(edittext.getSpanStart(myImageSpan));
      sortStartEnds.add(edittext.getSpanEnd(myImageSpan));
    sortStartEnds.add(edittext.length());
    Collections.sort(sortStartEnds);
    for (int i = 0; i < sortStartEnds.size(); i = i + 2) {
      start = sortStartEnds.get(i);
      end = sortStartEnds.get(i + 1);
      text = edittext.subSequence(start, end);
      if (!TextUtils.isEmpty(text)) {
        texts.add(new UnSpanText(start, end, text, spans[i].uid));
    return texts;
  @Override
  protected void performFiltering(CharSequence text, int keyCode) {
    super.performFiltering(text, keyCode);
  public void addSpan(String showText, String uid) {
    if (!TextUtils.isEmpty(getText().toString()) && getText().toString().lengt...
      WKToastUtils.getInstance()
          .showToast(getContext().getString(R.string.content_too_long));
      return;
    int index = getSelectionStart();
    getText().insert(index, showText);
    SpannableString spannableString = new SpannableString(getText());
    generateOneSpan(spannableString, new UnSpanText(index, index + showText.le...
    setText(spannableString);
    setSelection(index + showText.length());
  private void generateOneSpan(Spannable spannableString, UnSpanText unSpanText) {
    View spanView = getSpanView(getContext(), unSpanText.showText.toString(), ...
    BitmapDrawable bitmapDrawable = (BitmapDrawable) convertViewToDrawable(spa...
    bitmapDrawable.setBounds(0, 0, bitmapDrawable.getIntrinsicWidth(), bitmapD...
```

```
    MyImageSpan what = new MyImageSpan(bitmapDrawable, unSpanText.showText.toS...
    final int start = unSpanText.start;
    final int end = unSpanText.end;
    spannableString.setSpan(what, start, end, Spannable.SPAN_EXCLUSIVE_EXCLUSI...
  public Drawable convertViewToDrawable(View view) {
    int spec = View.MeasureSpec.makeMeasureSpec(0, View.MeasureSpec.UNSPECIFIED);
    view.measure(spec, spec);
    view.layout(0, 0, view.getMeasuredWidth(), view.getMeasuredHeight());
    Bitmap b = Bitmap.createBitmap(view.getMeasuredWidth(), view.getMeasuredHe...
    Canvas c = new Canvas(b);
    c.translate(-view.getScrollX(), -view.getScrollY());
    view.draw(c);
    view.setDrawingCacheEnabled(true);
    Bitmap cacheBmp = view.getDrawingCache();
    Bitmap viewBmp = cacheBmp.copy(Bitmap.Config.ARGB_8888, true);
    cacheBmp.recycle();
    view.destroyDrawingCache();
    return new BitmapDrawable(viewBmp);
  private Bitmap getBitmap(Context context, float size, String name) {
    Paint paint = new Paint();
    paint.setColor(context.getResources().getColor(R.color.black));
    paint.setAntiAlias(true);
    paint.setTextSize(size);
    Rect rect = new Rect();
    paint.getTextBounds(name, 0, name.length(), rect);
    int width = (int) (paint.measureText(name));
    int height = (int) (paint.getFontSpacing());
    Paint.FontMetricsInt fontMetrics = paint.getFontMetricsInt();
    float fontTotalHeight = fontMetrics.bottom - fontMetrics.top;
    Bitmap bmp = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_8888);
    Canvas canvas = new Canvas(bmp);
    float offY = fontTotalHeight / 2;
    float newY = rect.height() / 2 + offY;
    canvas.drawText(name, 0, newY - rect.bottom - 1, paint);
    return bmp;
  public View getSpanView(Context context, String text, int maxWidth) {
    TextView view = new TextView(context);
    view.setMaxWidth(maxWidth);
    view.setText(text);
    view.setEllipsize(TextUtils.TruncateAt.END);
    view.setSingleLine(true);
    view.setBackgroundResource(R.drawable.shape_corner_rectangle);
    view.setTextSize(getTextSize());
    setGravity(Gravity.CENTER_VERTICAL);
    view.setTextColor(ContextCompat.getColor(context, R.color.colorDark));
    FrameLayout frameLayout = new FrameLayout(context);
    frameLayout.addView(view);
    return frameLayout;
  private static class UnSpanText {
    int start;
```

```
    int end;
    CharSequence showText;
    String uid;
    UnSpanText(int start, int end, CharSequence showText, String uid) {
        this.start = start;
        this.end = end;
        this.showText = showText;
        this.uid = uid;
private static class MyImageSpan extends ImageSpan {
    private final String showText;
    private final String uid;
    public MyImageSpan(Drawable d, String showText, String uid) {
        super(d);
        this.showText = showText;
        this.uid = uid;
    public String getUid() {
        return uid;
    public String getShowText() {
        return showText;
public static int dip2px(Context context, int dip) {
    final float scale = context.getResources().getDisplayMetrics().density;
    return (int) (dip * scale + 0.5f);
public List<String> getAllUIDs() {
    List<String> uids = new ArrayList<>();
    Editable editText = getText();
    Spannable spannableString = new SpannableString(editText);
    MyImageSpan[] spans = spannableString.getSpans(0, editText.length(), MyIma...
    for (MyImageSpan span : spans) {
        uids.add(span.getUid());
    return uids;
@Override
public boolean onTextContextMenuItem(int id) {
    if (id == android.R.id.paste) {
        ClipboardManager cm = (ClipboardManager) getContext().getSystemService...
        assert cm != null;
        ClipData data = cm.getPrimaryClip();
        assert data != null;
        ClipData.Item item = data.getItemAt(0);
        String editContent = "";
        if (item.getText() != null)
            editContent = item.getText().toString();
        if (!TextUtils.isEmpty(editContent) && maxLength > 0) {
            int len = 0;
            if (!TextUtils.isEmpty(getText())) {
                len = getText().toString().length();
            if (len + editContent.length() > maxLength) {
                WKToastUtils.getInstance().showToast(getContext().getString(R....
                int addLen = maxLength - len;
                if (addLen > 0) {
                    editContent = editContent.substring(0, addLen);
```

```
            }else{
                return true;
        ClipboardManager clip = (ClipboardManager) getContext().getSystemServi...
        if (clip != null) {
            int curPosition = getSelectionStart();
            StringBuilder sb = new StringBuilder(Objects.requireNonNull(getTex...
            sb.insert(curPosition, editContent);
            this.setText(MoonUtil.getEmotionContent(getContext(), this, sb.toS...
            this.setSelection(curPosition + editContent.length());
            return true;
    return super.onTextContextMenuItem(id);
public List<WKMsgEntity> getAllEntity() {
    List<WKMsgEntity> list = new ArrayList<>();
    Spannable spannableString = new SpannableString(getEditableText());
    String spannedString = Objects.requireNonNull(getText()).toString();
    int next;
    for (int i = 0; i < spannableString.length(); i = next) {
        next = spannableString.nextSpanTransition(i, spannedString.length(), C...
        MyImageSpan[] mentionSpans = spannableString.getSpans(i, next, MyImage...
        for (MyImageSpan mentionSpan : mentionSpans) {
            WKMsgEntity entity = new WKMsgEntity();
            entity.type = ChatContentSpanType.getMention();
            entity.offset = i;
            entity.length = next - i;
            entity.value = mentionSpan.uid;
            list.add(entity);
    List<String> urls = StringUtils.getStrUrls(spannedString);
    for (String url : urls) {
        int fromIndex = 0;
        while (fromIndex >= 0) {
            fromIndex = spannedString.indexOf(url, fromIndex);
            if (fromIndex >= 0) {
                WKMsgEntity entity = new WKMsgEntity();
                entity.type = ChatContentSpanType.getLink();
                entity.offset = fromIndex;
                entity.length = url.length();
                entity.value = url;
                list.add(entity);
                fromIndex += url.length();
    List<String> emails = StringUtils.getEmails(spannedString);
    for (String email : emails) {
        int fromIndex = 0;
        while (fromIndex >= 0) {
            fromIndex = spannedString.indexOf(email, fromIndex);
            if (fromIndex >= 0) {
                WKMsgEntity entity = new WKMsgEntity();
                entity.type = ChatContentSpanType.getLink();
                entity.offset = fromIndex;
                entity.length = email.length();
                entity.value = email;
```

```
                list.add(entity);
                fromIndex += email.length();
        List<String> phones = StringUtils.getNumbers(spannedString);
        for (String phone : phones) {
            int fromIndex = 0;
            while (fromIndex >= 0) {
                fromIndex = spannedString.indexOf(phone, fromIndex);
                if (fromIndex >= 0) {
                    WKMsgEntity entity = new WKMsgEntity();
                    entity.type = ChatContentSpanType.getLink();
                    entity.offset = fromIndex;
                    entity.length = phone.length();
                    entity.value = phone;
                    list.add(entity);
                    fromIndex += phone.length();
        return list;
public class Scroller  {
    private int mMode;
    private int mStartX;
    private int mStartY;
    private int mFinalX;
    private int mFinalY;
    private int mMinX;
    private int mMaxX;
    private int mMinY;
    private int mMaxY;
    private int mCurrX;
    private int mCurrY;
    private long mStartTime;
    private int mDuration;
    private float mDurationReciprocal;
    private float mDeltaX;
    private float mDeltaY;
    private boolean mFinished;
    private Interpolator mInterpolator;
    private boolean mFlywheel;
    private float mVelocity;
    private static final int DEFAULT_DURATION = 250;
    private static final int SCROLL_MODE = 0;
    private static final int FLING_MODE = 1;
    private static float DECELERATION_RATE = (float) (Math.log(0.75) / Math.log(0....
    private static float START_TENSION = 0.4f;
    private static float END_TENSION = 1.0f - START_TENSION;
    private static final int NB_SAMPLES = 100;
    private static final float[] SPLINE = new float[NB_SAMPLES + 1];
    private float mDeceleration;
    private final float mPpi;
    static {
        float x_min = 0.0f;
        for (int i = 0; i <= NB_SAMPLES; i++) {
```

```
        final float t = (float) i / NB_SAMPLES;
        float x_max = 1.0f;
        float x, tx, coef;
        while (true) {
            x = x_min + (x_max - x_min) / 2.0f;
            coef = 3.0f * x * (1.0f - x);
            tx = coef * ((1.0f - x) * START_TENSION + x * END_TENSION) + x * x...
            if (Math.abs(tx - t) < 1E-5) break;
            if (tx > t) x_max = x;
            else x_min = x;
        final float d = coef + x * x * x;
        SPLINE[i] = d;
    SPLINE[NB_SAMPLES] = 1.0f;
    sViscousFluidScale = 8.0f;
    sViscousFluidNormalize = 1.0f;
    sViscousFluidNormalize = 1.0f / viscousFluid(1.0f);
private static float sViscousFluidScale;
private static float sViscousFluidNormalize;
public Scroller(Context context) {
    this(context, null);
public Scroller(Context context, Interpolator interpolator) {
    this(context, interpolator, true);
public Scroller(Context context, Interpolator interpolator, boolean flywheel) {
    mFinished = true;
    mInterpolator = interpolator;
    mPpi = context.getResources().getDisplayMetrics().density * 160.0f;
    mDeceleration = computeDeceleration(ViewConfiguration.getScrollFriction());
    mFlywheel = flywheel;
public final void setFriction(float friction) {
    mDeceleration = computeDeceleration(friction);
private float computeDeceleration(float friction) {
    return SensorManager.GRAVITY_EARTH
public final boolean isFinished() {
    return mFinished;
public final void forceFinished(boolean finished) {
    mFinished = finished;
public final int getDuration() {
    return mDuration;
public final int getCurrX() {
    return mCurrX;
public final int getCurrY() {
    return mCurrY;
public float getCurrVelocity() {
    return mVelocity - mDeceleration * timePassed() / 2000.0f;
public final int getStartX() {
    return mStartX;
public final int getStartY() {
    return mStartY;
public final int getFinalX() {
    return mFinalX;
```

```
public final int getFinalY() {
    return mFinalY;
public boolean computeScrollOffset() {
    if (mFinished) {
        return false;
    int timePassed = (int)(AnimationUtils.currentAnimationTimeMillis() - mStar...
    if (timePassed < mDuration) {
        switch (mMode) {
        case SCROLL_MODE:
            float x = timePassed * mDurationReciprocal;
            if (mInterpolator == null)
                x = viscousFluid(x);
            else
                x = mInterpolator.getInterpolation(x);
            mCurrX = mStartX + Math.round(x * mDeltaX);
            mCurrY = mStartY + Math.round(x * mDeltaY);
            break;
        case FLING_MODE:
            final float t = (float) timePassed / mDuration;
            final int index = (int) (NB_SAMPLES * t);
            final float t_inf = (float) index / NB_SAMPLES;
            final float t_sup = (float) (index + 1) / NB_SAMPLES;
            final float d_inf = SPLINE[index];
            final float d_sup = SPLINE[index + 1];
            final float distanceCoef = d_inf + (t - t_inf) / (t_sup - t_inf) *...
            mCurrX = mStartX + Math.round(distanceCoef * (mFinalX - mStartX));
            mCurrX = Math.min(mCurrX, mMaxX);
            mCurrX = Math.max(mCurrX, mMinX);
            mCurrY = mStartY + Math.round(distanceCoef * (mFinalY - mStartY));
            mCurrY = Math.min(mCurrY, mMaxY);
            mCurrY = Math.max(mCurrY, mMinY);
            if (mCurrX == mFinalX && mCurrY == mFinalY) {
                mFinished = true;
            break;
    else {
        mCurrX = mFinalX;
        mCurrY = mFinalY;
        mFinished = true;
    return true;
public void startScroll(int startX, int startY, int dx, int dy) {
    startScroll(startX, startY, dx, dy, DEFAULT_DURATION);
public void startScroll(int startX, int startY, int dx, int dy, int duration) {
    mMode = SCROLL_MODE;
    mFinished = false;
    mDuration = duration;
    mStartTime = AnimationUtils.currentAnimationTimeMillis();
    mStartX = startX;
    mStartY = startY;
    mFinalX = startX + dx;
    mFinalY = startY + dy;
```

```
    mDeltaX = dx;
    mDeltaY = dy;
    mDurationReciprocal = 1.0f / (float) mDuration;
  public void fling(int startX, int startY, int velocityX, int velocityY,
        int minX, int maxX, int minY, int maxY) {
    if (mFlywheel && !mFinished) {
      float oldVel = getCurrVelocity();
      float dx = (float) (mFinalX - mStartX);
      float dy = (float) (mFinalY - mStartY);
      float hyp = (float) Math.sqrt(dx * dx + dy * dy);
      float ndx = dx / hyp;
      float ndy = dy / hyp;
      float oldVelocityX = ndx * oldVel;
      float oldVelocityY = ndy * oldVel;
      if (Math.signum(velocityX) == Math.signum(oldVelocityX) &&
          Math.signum(velocityY) == Math.signum(oldVelocityY)) {
        velocityX += oldVelocityX;
        velocityY += oldVelocityY;
    mMode = FLING_MODE;
    mFinished = false;
    float velocity = (float) Math.sqrt(velocityX * velocityX + velocityY * vel...
    mVelocity = velocity;
    float ALPHA = 800;
    final double l = Math.log(START_TENSION * velocity / ALPHA);
    mDuration = (int) (1000.0 * Math.exp(l / (DECELERATION_RATE - 1.0)));
    mStartTime = AnimationUtils.currentAnimationTimeMillis();
    mStartX = startX;
    mStartY = startY;
    float coeffX = velocity == 0 ? 1.0f : velocityX / velocity;
    float coeffY = velocity == 0 ? 1.0f : velocityY / velocity;
    int totalDistance =
        (int) (ALPHA * Math.exp(DECELERATION_RATE / (DECELERATION_RATE - 1...
    mMinX = minX;
    mMaxX = maxX;
    mMinY = minY;
    mMaxY = maxY;
    mFinalX = startX + Math.round(totalDistance * coeffX);
    mFinalX = Math.min(mFinalX, mMaxX);
    mFinalX = Math.max(mFinalX, mMinX);
    mFinalY = startY + Math.round(totalDistance * coeffY);
    mFinalY = Math.min(mFinalY, mMaxY);
    mFinalY = Math.max(mFinalY, mMinY);
  static float viscousFluid(float x)
    x *= sViscousFluidScale;
    if (x < 1.0f) {
      x -= (1.0f - (float)Math.exp(-x));
    } else {
      float start = 0.36787944117f;
      x = 1.0f - (float)Math.exp(1.0f - x);
      x = start + x * (1.0f - start);
```

```
        x *= sViscousFluidNormalize;
        return x;
    public void abortAnimation() {
        mCurrX = mFinalX;
        mCurrY = mFinalY;
        mFinished = true;
    public void extendDuration(int extend) {
        int passed = timePassed();
        mDuration = passed + extend;
        mDurationReciprocal = 1.0f / mDuration;
        mFinished = false;
    public int timePassed() {
        return (int)(AnimationUtils.currentAnimationTimeMillis() - mStartTime);
    public void setFinalX(int newX) {
        mFinalX = newX;
        mDeltaX = mFinalX - mStartX;
        mFinished = false;
    public void setFinalY(int newY) {
        mFinalY = newY;
        mDeltaY = mFinalY - mStartY;
        mFinished = false;
    public boolean isScrollingInDirection(float xvel, float yvel) {
        return !mFinished && Math.signum(xvel) == Math.signum(mFinalX - mStartX) &&
                Math.signum(yvel) == Math.signum(mFinalY - mStartY);
public class TypeWriterView extends AppCompatTextView {
    private CharSequence mText;
    private String mPrintingText;
    private int mIndex;
    private final long mDelay = 100;
    private Context mContext;
    private boolean animating = false;
    private Runnable mBlinker;
    private int i = 0;
    private final Handler mHandler = new Handler();
    private final Runnable mCharacterAdder = new Runnable() {
        @Override
        public void run() {
            if (animating) {
                setText(String.format("%s _", mText.subSequence(0, mIndex++)));
                if (mIndex <= mText.length()) {
                    mHandler.postDelayed(mCharacterAdder, mDelay);
                } else {
                    animating = false;
                    callBlink();
    public TypeWriterView(Context context) {
        super(context);
        mContext = context;
    public TypeWriterView(Context context, AttributeSet attrs) {
        super(context, attrs);
    public void append(String str) {
```

```
        mText = mText + str;
        mPrintingText = mPrintingText + str;
        if (!animating) {
            mHandler.removeCallbacks(mCharacterAdder);
            animating = true;
            mHandler.removeCallbacks(mBlinker);
            mHandler.postDelayed(mCharacterAdder, mDelay);
    private void callBlink() {
        mBlinker = new Runnable() {
            @Override
            public void run() {
                if (i <= 10) {
                    if (i++ % 2 != 0) {
                        setText(String.format("%s _", mText));
                        mHandler.postDelayed(mBlinker, 150);
                    } else {
                        setText(String.format("%s  ", mText));
                        mHandler.postDelayed(mBlinker, 150);
                } else
                    i = 0;
        mHandler.postDelayed(mBlinker, 150);
    public void animateText(String text) {
        if (!animating) {
            animating = true;
            mText = text;
            mPrintingText = text;
            mIndex = 0;
            setText("");
            mHandler.removeCallbacks(mCharacterAdder);
            mHandler.postDelayed(mCharacterAdder, mDelay);
        } else {
            Toast.makeText(mContext, "Typewriter busy typing: " + mText, Toast.LEN...
    public void removeAnimation() {
        mHandler.removeCallbacks(mCharacterAdder);
        animating = false;
        setText(mPrintingText);
public abstract class SeekBarAccessibilityDelegate extends View.AccessibilityDeleg...
    private static final CharSequence SEEK_BAR_CLASS_NAME = SeekBar.class.getName();
    private final Map<View, Runnable> accessibilityEventRunnables = new HashMap<>(4);
    private final View.OnAttachStateChangeListener onAttachStateChangeListener = n...
        @Override
        public void onViewAttachedToWindow(View v) {
        @Override
        public void onViewDetachedFromWindow(View v) {
            v.removeCallbacks(accessibilityEventRunnables.remove(v));
            v.removeOnAttachStateChangeListener(this);
    @Override
    public boolean performAccessibilityAction(@NonNull View host, int action, Bund...
        if (super.performAccessibilityAction(host, action, args)) {
            return true;
```

```
        return performAccessibilityActionInternal(host, action, args);
    public boolean performAccessibilityActionInternal(@Nullable View host, int act...
        if (action == AccessibilityNodeInfo.ACTION_SCROLL_FORWARD || action == Acc...
            doScroll(host, action == AccessibilityNodeInfo.ACTION_SCROLL_BACKWARD);
            if (host != null) {
                postAccessibilityEventRunnable(host);
            return true;
        return false;
    public final boolean performAccessibilityActionInternal(int action, Bundle arg...
        return performAccessibilityActionInternal(null, action, args);
    private void postAccessibilityEventRunnable(@NonNull View host) {
        if (!ViewCompat.isAttachedToWindow(host)) {
            return;
        Runnable runnable = accessibilityEventRunnables.get(host);
        if (runnable == null) {
            accessibilityEventRunnables.put(host, runnable = () -> sendAccessibili...
            host.addOnAttachStateChangeListener(onAttachStateChangeListener);
        } else {
            host.removeCallbacks(runnable);
        host.postDelayed(runnable, 400);
    @Override
    public void onInitializeAccessibilityNodeInfo(@NonNull View host, Accessibilit...
        super.onInitializeAccessibilityNodeInfo(host, info);
        onInitializeAccessibilityNodeInfoInternal(host, info);
    public void onInitializeAccessibilityNodeInfoInternal(@Nullable View host, Acc...
        info.setClassName(SEEK_BAR_CLASS_NAME);
        final CharSequence contentDescription = getContentDescription(host);
        if (!TextUtils.isEmpty(contentDescription)) {
            info.setText(contentDescription);
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP) {
            if (canScrollBackward(host)) {
                info.addAction(AccessibilityNodeInfo.AccessibilityAction.ACTION_SC...
            if (canScrollForward(host)) {
                info.addAction(AccessibilityNodeInfo.AccessibilityAction.ACTION_SC...
    public final void onInitializeAccessibilityNodeInfoInternal(AccessibilityNodeI...
        onInitializeAccessibilityNodeInfoInternal(null, info);
    protected CharSequence getContentDescription(@Nullable View host) {
        return null;
    protected abstract void doScroll(@Nullable View host, boolean backward);
    protected abstract boolean canScrollBackward(@Nullable View host);
    protected abstract boolean canScrollForward(@Nullable View host);
public class RoundTextView extends AppCompatTextView {
    int borderWidth = AndroidUtilities.dp(1);
    int borderColor = 0;
    float radius = 0f;
    float topLeftRadius = 0f;
    float topRightRadius = 0f;
    float bottomLeftRadius = 0f;
    float bottomRightRadius = 0f;
    int backgroundColor = 0;
```

```
    public RoundTextView(Context context) {
        super(context);
        reset();
    public RoundTextView(Context context, @Nullable AttributeSet attrs) {
        super(context, attrs);
        init(context, attrs);
    public RoundTextView(Context context, @Nullable AttributeSet attrs, int defSty...
        super(context, attrs, defStyleAttr);
        init(context, attrs);
    private void init(Context context, AttributeSet attrs) {
        TypedArray attributes = context.obtainStyledAttributes(attrs, R.styleable....
        if (attributes != null) {
            borderWidth = attributes.getDimensionPixelSize(R.styleable.RoundTextVi...
            borderColor = attributes.getColor(R.styleable.RoundTextView_tvBorderCo...
            radius = attributes.getDimension(R.styleable.RoundTextView_tvCornerRad...
            topLeftRadius = attributes.getDimension(R.styleable.RoundTextView_tvTo...
            topRightRadius = attributes.getDimension(R.styleable.RoundTextView_tvT...
            bottomLeftRadius = attributes.getDimension(R.styleable.RoundTextView_t...
            bottomRightRadius = attributes.getDimension(R.styleable.RoundTextView_...
            backgroundColor = attributes.getColor(R.styleable.RoundTextView_tvBack...
            attributes.recycle();
            reset();
    private void reset() {
        GradientDrawable gd = new GradientDrawable();
        gd.setColor(backgroundColor);
        if (borderWidth > 0) {
            gd.setStroke(borderWidth, borderColor);
        if (topLeftRadius > 0 || topRightRadius > 0 || bottomLeftRadius > 0 || bot...
            float[] radii = new float[]{
                    topLeftRadius, topLeftRadius,
                    topRightRadius, topRightRadius,
                    bottomRightRadius, bottomRightRadius,
                    bottomLeftRadius, bottomLeftRadius
            gd.setCornerRadii(radii);
        } else {
            gd.setCornerRadius(radius);
        this.setBackground(gd);
    public void setBackGroundColor(int color) {
        GradientDrawable myGrad = (GradientDrawable) getBackground();
        myGrad.setColor(color);
        backgroundColor = color;
    public void setBorderColor(int color) {
        borderColor = color;
        borderWidth = AndroidUtilities.dp(1);
        reset();
    public void setAllRadius(float radius) {
        topLeftRadius = topRightRadius = bottomRightRadius = bottomLeftRadius = An...
        reset();
public class PopupSwipeBackLayout extends FrameLayout {
    private final static int DURATION = 300;
```

```
SparseIntArray overrideHeightIndex = new SparseIntArray();
public float transitionProgress;
private float toProgress = -1;
private GestureDetectorCompat detector;
private boolean isProcessingSwipe;
private boolean isAnimationInProgress;
private boolean isSwipeDisallowed;
private Paint overlayPaint = new Paint(Paint.ANTI_ALIAS_FLAG);
private Paint foregroundPaint = new Paint();
private int foregroundColor = 0;
private Path mPath = new Path();
private RectF mRect = new RectF();
private ArrayList<OnSwipeBackProgressListener> onSwipeBackProgressListeners = ...
private boolean isSwipeBackDisallowed;
private float overrideForegroundHeight;
private ValueAnimator foregroundAnimator;
private int currentForegroundIndex = -1;
private Rect hitRect = new Rect();
public PopupSwipeBackLayout(@NonNull Context context) {
  super(context);
  int touchSlop = ViewConfiguration.get(context).getScaledTouchSlop();
  detector = new GestureDetectorCompat(context, new GestureDetector.SimpleOn...
    @Override
    public boolean onDown(MotionEvent e) {
      return true;
    @Override
    public boolean onScroll(MotionEvent e1, MotionEvent e2, float distance...
      if (!isProcessingSwipe && !isSwipeDisallowed) {
        if (!isSwipeBackDisallowed && transitionProgress == 1 && dista...
          isProcessingSwipe = true;
          MotionEvent c = MotionEvent.obtain(0, 0, MotionEvent.ACTIO...
          for (int i = 0; i < getChildCount(); i++)
            getChildAt(i).dispatchTouchEvent(c);
          c.recycle();
        } else isSwipeDisallowed = true;
      if (isProcessingSwipe) {
        toProgress = -1;
        transitionProgress = 1f - Math.max(0, Math.min(1, (e2.getX() -...
        invalidateTransforms();
      return isProcessingSwipe;
    @Override
    public boolean onFling(MotionEvent e1, MotionEvent e2, float velocityX...
      if (isAnimationInProgress || isSwipeDisallowed)
        return false;
      if (velocityX >= 600) {
        clearFlags();
        animateToState(0, velocityX / 6000f);
      return false;
  });
  overlayPaint.setColor(Color.BLACK);
```

```
public void setSwipeBackDisallowed(boolean swipeBackDisallowed) {
    isSwipeBackDisallowed = swipeBackDisallowed;
public void addOnSwipeBackProgressListener(OnSwipeBackProgressListener onSwipe...
    onSwipeBackProgressListeners.add(onSwipeBackProgressListener);
@Override
protected boolean drawChild(Canvas canvas, View child, long drawingTime) {
    int i = indexOfChild(child);
    int s = canvas.save();
    if (i != 0) {
        if (foregroundColor == 0) {
            foregroundPaint.setColor(Theme.colorAccount);
        } else {
            foregroundPaint.setColor(foregroundColor);
        canvas.drawRect(child.getX(), 0, child.getX() + child.getMeasuredWidth...
    boolean b = super.drawChild(canvas, child, drawingTime);
    if (i == 0) {
        overlayPaint.setAlpha((int) (transitionProgress * 0x40));
        canvas.drawRect(0, 0, getWidth(), getHeight(), overlayPaint);
    canvas.restoreToCount(s);
    return b;
public void invalidateTransforms() {
    if (!onSwipeBackProgressListeners.isEmpty()) {
        for (int i = 0; i < onSwipeBackProgressListeners.size(); i++) {
            onSwipeBackProgressListeners.get(i).onSwipeBackProgress(this, toPr...
    View backgroundView = getChildAt(0);
    View foregroundView = null;
    if (currentForegroundIndex >= 0 && currentForegroundIndex < getChildCount(...
        foregroundView = getChildAt(currentForegroundIndex);
    backgroundView.setTranslationX(-transitionProgress * getWidth() * 0.5f);
    float bSc = 0.95f + (1f - transitionProgress) * 0.05f;
    backgroundView.setScaleX(bSc);
    backgroundView.setScaleY(bSc);
    if (foregroundView != null) {
        foregroundView.setTranslationX((1f - transitionProgress) * getWidth());
    invalidateVisibility();
    float fW = backgroundView.getMeasuredWidth(), fH = backgroundView.getMeasu...
    float tW = 0;
    float tH = 0;
    if (foregroundView != null) {
        tW = foregroundView.getMeasuredWidth();
        tH = overrideForegroundHeight != 0 ? overrideForegroundHeight : foregr...
    if (backgroundView.getMeasuredWidth() == 0 || backgroundView.getMeasuredHe...
        return;
    ActionBarPopupWindow.ActionBarPopupWindowLayout p = (ActionBarPopupWindow....
    float w = fW + (tW - fW) * transitionProgress;
    float h = fH + (tH - fH) * transitionProgress;
    w += p.getPaddingLeft() + p.getPaddingRight();
    h += p.getPaddingTop() + p.getPaddingBottom();
    p.updateAnimation = false;
    p.setBackScaleX(w / p.getMeasuredWidth());
```

```
    p.setBackScaleY(h / p.getMeasuredHeight());
    p.updateAnimation = true;
    for (int i = 0; i < getChildCount(); i++) {
      View ch = getChildAt(i);
      ch.setPivotX(0);
      ch.setPivotY(0);
    invalidate();
  @Override
  public boolean dispatchTouchEvent(MotionEvent ev) {
    if (processTouchEvent(ev))
      return true;
    int act = ev.getActionMasked();
    if (act == MotionEvent.ACTION_DOWN && !mRect.contains(ev.getX(), ev.getY()...
      callOnClick();
      return true;
    if (currentForegroundIndex < 0 || currentForegroundIndex >= getChildCount(...
      return super.dispatchTouchEvent(ev);
    View bv = getChildAt(0);
    View fv = getChildAt(currentForegroundIndex);
    boolean b = (transitionProgress > 0.5f ? fv : bv).dispatchTouchEvent(ev);
    if (!b && act == MotionEvent.ACTION_DOWN) {
      return true;
    return b || onTouchEvent(ev);
  @Override
  protected void onSizeChanged(int w, int h, int oldw, int oldh) {
    super.onSizeChanged(w, h, oldw, oldh);
    invalidateTransforms();
  private boolean processTouchEvent(MotionEvent ev) {
    int act = ev.getAction() & MotionEvent.ACTION_MASK;
    if (isAnimationInProgress)
      return true;
    if (!detector.onTouchEvent(ev)) {
      switch (act) {
        case MotionEvent.ACTION_DOWN:
          break;
        case MotionEvent.ACTION_CANCEL:
        case MotionEvent.ACTION_UP:
          if (isProcessingSwipe) {
            clearFlags();
            animateToState(transitionProgress >= 0.5f ? 1 : 0, 0);
          } else if (isSwipeDisallowed) clearFlags();
          return false;
    return isProcessingSwipe;
  private void animateToState(float f, float flingVal) {
    ValueAnimator val = ValueAnimator.ofFloat(transitionProgress, f).setDurati...
    val.setInterpolator(CubicBezierInterpolator.DEFAULT);
    int selectedAccount = 1;
    val.addUpdateListener(animation -> {
      transitionProgress = (float) animation.getAnimatedValue();
      invalidateTransforms();
```

```
    });
    val.addListener(new AnimatorListenerAdapter() {
        @Override
        public void onAnimationStart(Animator animation) {
            isAnimationInProgress = true;
            toProgress = f;
        @Override
        public void onAnimationEnd(Animator animation) {
            transitionProgress = f;
            invalidateTransforms();
            isAnimationInProgress = false;
    });
    val.start();
private void clearFlags() {
    isProcessingSwipe = false;
    isSwipeDisallowed = false;
public void openForeground(int viewIndex) {
    if (isAnimationInProgress) {
        return;
    currentForegroundIndex = viewIndex;
    overrideForegroundHeight = overrideHeightIndex.get(viewIndex);
    animateToState(1, 0);
public void closeForeground() {
    closeForeground(true);
public void closeForeground(boolean animated) {
    if (isAnimationInProgress) return;
    if (!animated) {
        currentForegroundIndex = -1;
        transitionProgress = 0;
        invalidateTransforms();
        return;
    } else {
        animateToState(0, 0);
@Override
protected void onLayout(boolean changed, int left, int top, int right, int bot...
    for (int i = 0; i < getChildCount(); i++) {
        View ch = getChildAt(i);
        ch.layout(0, 0, ch.getMeasuredWidth(), ch.getMeasuredHeight());
@Override
public void addView(View child, int index, ViewGroup.LayoutParams params) {
    super.addView(child, index, params);
    invalidateTransforms();
@Override
protected void dispatchDraw(Canvas canvas) {
    if (getChildCount() == 0) {
        return;
    View backgroundView = getChildAt(0);
    float fW = backgroundView.getMeasuredWidth(), fH = backgroundView.getMeasu...
    float w, h;
    if (currentForegroundIndex == -1 || currentForegroundIndex >= getChildCoun...
```

```
            w = fW;
            h = fH;
        } else {
            View foregroundView = getChildAt(currentForegroundIndex);
            float tW = foregroundView.getMeasuredWidth(), tH = overrideForegroundH...
            if (backgroundView.getMeasuredWidth() == 0 || backgroundView.getMeasur...
                w = fW;
                h = fH;
            } else {
                w = fW + (tW - fW) * transitionProgress;
                h = fH + (tH - fH) * transitionProgress;
        int s = canvas.save();
        mPath.rewind();
        int rad = AndroidUtilities.dp(6);
        mRect.set(0, 0, w, h);
        mPath.addRoundRect(mRect, rad, rad, Path.Direction.CW);
        canvas.clipPath(mPath);
        super.dispatchDraw(canvas);
        canvas.restoreToCount(s);
    private boolean isDisallowedView(MotionEvent e, View v) {
        v.getHitRect(hitRect);
        if (hitRect.contains((int) e.getX(), (int) e.getY()) && v.canScrollHorizon...
            return true;
        if (v instanceof ViewGroup) {
            ViewGroup vg = (ViewGroup) v;
            for (int i = 0; i < vg.getChildCount(); i++)
                if (isDisallowedView(e, vg.getChildAt(i)))
                    return true;
        return false;
    private void invalidateVisibility() {
        for (int i = 0; i < getChildCount(); i++) {
            View child = getChildAt(i);
            if (i == 0) {
                if (transitionProgress == 1 && child.getVisibility() != INVISIBLE)
                    child.setVisibility(INVISIBLE);
                if (transitionProgress != 1 && child.getVisibility() != VISIBLE)
                    child.setVisibility(VISIBLE);
            } else if (i == currentForegroundIndex) {
                if (transitionProgress == 0 && child.getVisibility() != INVISIBLE)
                    child.setVisibility(INVISIBLE);
                if (transitionProgress != 0 && child.getVisibility() != VISIBLE)
                    child.setVisibility(VISIBLE);
            } else {
                child.setVisibility(INVISIBLE);
    public void setNewForegroundHeight(int index, int height) {
        overrideHeightIndex.put(index, height);
        if (index != currentForegroundIndex) {
            return;
        if (currentForegroundIndex < 0 || currentForegroundIndex >= getChildCount(...
            return;
```

```
        if (foregroundAnimator != null) {
            foregroundAnimator.cancel();
        View fg = getChildAt(currentForegroundIndex);
        float fromH = overrideForegroundHeight != 0 ? overrideForegroundHeight : f...
        float toH = height;
        ValueAnimator animator = ValueAnimator.ofFloat(fromH, toH).setDuration(240);
        animator.setInterpolator(new TimeInterpolator() {
            @Override
            public float getInterpolation(float v) {
                return 0;
        });
        animator.addUpdateListener(animation -> {
            overrideForegroundHeight = (float) animation.getAnimatedValue();
            invalidateTransforms();
        });
        animator.addListener(new AnimatorListenerAdapter() {
            @Override
            public void onAnimationEnd(Animator animation) {
                isAnimationInProgress = false;
            @Override
            public void onAnimationStart(Animator animation) {
                isAnimationInProgress = true;
        });
        animator.start();
        foregroundAnimator = animator;
    public void setForegroundColor(int color) {
        foregroundColor = color;
    public interface OnSwipeBackProgressListener {
        void onSwipeBackProgress(PopupSwipeBackLayout layout, float toProgress, fl...
public class CubicBezierInterpolator implements Interpolator {
    public static final CubicBezierInterpolator DEFAULT = new CubicBezierInterpola...
    public static final CubicBezierInterpolator EASE_OUT = new CubicBezierInterpol...
    public static final CubicBezierInterpolator EASE_OUT_QUINT = new CubicBezierIn...
    public static final CubicBezierInterpolator EASE_IN = new CubicBezierInterpola...
    public static final CubicBezierInterpolator EASE_BOTH = new CubicBezierInterpo...
    protected PointF start;
    protected PointF end;
    protected PointF a = new PointF();
    protected PointF b = new PointF();
    protected PointF c = new PointF();
    public CubicBezierInterpolator(PointF start, PointF end) throws IllegalArgumen...
        if (start.x < 0 || start.x > 1) {
            throw new IllegalArgumentException("startX value must be in the range ...
        if (end.x < 0 || end.x > 1) {
            throw new IllegalArgumentException("endX value must be in the range [0...
        this.start = start;
        this.end = end;
    public CubicBezierInterpolator(float startX, float startY, float endX, float e...
        this(new PointF(startX, startY), new PointF(endX, endY));
    public CubicBezierInterpolator(double startX, double startY, double endX, doub...
```

```
        this((float) startX, (float) startY, (float) endX, (float) endY);
    @Override
    public float getInterpolation(float time) {
        return getBezierCoordinateY(getXForTime(time));
    protected float getBezierCoordinateY(float time) {
        c.y = 3 * start.y;
        b.y = 3 * (end.y - start.y) - c.y;
        a.y = 1 - c.y - b.y;
        return time * (c.y + time * (b.y + time * a.y));
    protected float getXForTime(float time) {
        float x = time;
        float z;
        for (int i = 1; i < 14; i++) {
            z = getBezierCoordinateX(x) - time;
            if (Math.abs(z) < 1e-3) {
                break;
            x -= z / getXDerivate(x);
        return x;
    private float getXDerivate(float t) {
        return c.x + t * (2 * b.x + 3 * a.x * t);
    private float getBezierCoordinateX(float time) {
        c.x = 3 * start.x;
        b.x = 3 * (end.x - start.x) - c.x;
        a.x = 1 - c.x - b.x;
        return time * (c.x + time * (b.x + time * a.x));
public class CounterView extends View {
    public CounterDrawable counterDrawable;
    public CounterView(Context context, AttributeSet attrs) {
        super(context, attrs);
        initView();
    public CounterView(Context context, AttributeSet attrs, int defStyle) {
        super(context, attrs, defStyle);
        initView();
    public CounterView(Context context) {
        super(context);
        initView();
    private void initView() {
        setVisibility(View.VISIBLE);
        counterDrawable = new CounterDrawable(this, true);
        counterDrawable.updateVisibility = true;
    @Override
    protected void onMeasure(int widthMeasureSpec, int heightMeasureSpec) {
        super.onMeasure(widthMeasureSpec, heightMeasureSpec);
        counterDrawable.setSize(getMeasuredHeight(), getMeasuredWidth());
    @Override
    protected void onDraw(Canvas canvas) {
        counterDrawable.draw(canvas);
    public void setColors(int textColor, int circleColor) {
        counterDrawable.textColor = textColor;
        counterDrawable.circleColor = circleColor;
```

```
    invalidate();
  public void setGravity(int gravity) {
    counterDrawable.gravity = gravity;
  public void setReverse(boolean b) {
    counterDrawable.reverseAnimation = b;
  public void setCount(int count, boolean animated) {
    counterDrawable.setCount(count, animated);
  public int getCount() {
    return counterDrawable.currentCount;
  public static class CounterDrawable {
    private final static int ANIMATION_TYPE_IN = 0;
    private final static int ANIMATION_TYPE_OUT = 1;
    private final static int ANIMATION_TYPE_REPLACE = 2;
    public boolean shortFormat = true;
    int animationType = -1;
    public Paint circlePaint;
    public TextPaint textPaint = new TextPaint(Paint.ANTI_ALIAS_FLAG);
    public RectF rectF = new RectF();
    public boolean addServiceGradient;
    int currentCount;
    private boolean countAnimationIncrement;
    private ValueAnimator countAnimator;
    public float countChangeProgress = 1f;
    private StaticLayout countLayout;
    private StaticLayout countOldLayout;
    private StaticLayout countAnimationStableLayout;
    private StaticLayout countAnimationInLayout;
    private int countWidthOld;
    private int countWidth;
    public int textColor = R.color.white;
    public int circleColor = Theme.colorAccount;
    int lastH;
    int width;
    public int gravity = Gravity.CENTER;
    float countLeft;
    float x;
    private boolean reverseAnimation;
    public float horizontalPadding;
    private boolean drawBackground = true;
    public boolean updateVisibility;
    private View parent;
    public final static int TYPE_DEFAULT = 0;
    public final static int TYPE_CHAT_PULLING_DOWN = 1;
    public final static int TYPE_CHAT_REACTIONS = 2;
    int type = TYPE_DEFAULT;
    public CounterDrawable(View parent, boolean drawBackground) {
      this.parent = parent;
      this.drawBackground = drawBackground;
      if (drawBackground) {
        circlePaint = new Paint(Paint.ANTI_ALIAS_FLAG);
```

```
        circlePaint.setColor(Color.BLACK);
    textPaint.setTextSize(AndroidUtilities.dp(12));
public void setSize(int h, int w) {
    if (h != lastH) {
        int count = currentCount;
        currentCount = -1;
        setCount(count, animationType == ANIMATION_TYPE_IN);
        lastH = h;
    width = w;
private void drawInternal(Canvas canvas) {
    float countTop = (lastH - AndroidUtilities.dp(20)) / 2f;
    updateX(countWidth);
    rectF.set(x, countTop, x + countWidth + AndroidUtilities.dp(11), count...
    if (circlePaint != null && drawBackground) {
        canvas.drawRoundRect(rectF, 11.5f * AndroidUtilities.density, 11.5...
        if (addServiceGradient) {
            canvas.drawRoundRect(rectF, 11.5f * AndroidUtilities.density, ...
    if (countLayout != null) {
        canvas.save();
        canvas.translate(countLeft, countTop + AndroidUtilities.dp(4));
        countLayout.draw(canvas);
        canvas.restore();
public void setCount(int count, boolean animated) {
    if (count == currentCount) {
        return;
    if (countAnimator != null) {
        countAnimator.cancel();
    if (count > 0 && updateVisibility && parent != null) {
        parent.setVisibility(View.VISIBLE);
    if (Math.abs(count - currentCount) > 99) {
        animated = false;
    if (!animated) {
        currentCount = count;
        if (count == 0) {
            if (updateVisibility && parent != null) {
                parent.setVisibility(View.GONE);
            return;
        String newStr = getStringOfCCount(count);
        countWidth = Math.max(AndroidUtilities.dp(12), (int) Math.ceil(tex...
        countLayout = new StaticLayout(newStr, textPaint, countWidth, Layo...
        if (parent != null) {
            parent.invalidate();
    String newStr = getStringOfCCount(count);
    if (animated) {
        if (countAnimator != null) {
            countAnimator.cancel();
        countChangeProgress = 0f;
        countAnimator = ValueAnimator.ofFloat(0, 1f);
        countAnimator.addUpdateListener(valueAnimator -> {
            countChangeProgress = (float) valueAnimator.getAnimatedValue();
```

```
            if (parent != null) {
                parent.invalidate();
            }
        });
        countAnimator.addListener(new AnimatorListenerAdapter() {
            @Override
            public void onAnimationEnd(Animator animation) {
                countChangeProgress = 1f;
                countOldLayout = null;
                countAnimationStableLayout = null;
                countAnimationInLayout = null;
                if (parent != null) {
                    if (currentCount == 0 && updateVisibility) {
                        parent.setVisibility(View.GONE);
                    parent.invalidate();
                animationType = -1;
        });
        if (currentCount <= 0) {
            animationType = ANIMATION_TYPE_IN;
            countAnimator.setDuration(220);
            countAnimator.setInterpolator(new OvershootInterpolator());
        } else if (count == 0) {
            animationType = ANIMATION_TYPE_OUT;
            countAnimator.setDuration(150);
            countAnimator.setInterpolator(CubicBezierInterpolator.DEFAULT);
        } else {
            animationType = ANIMATION_TYPE_REPLACE;
            countAnimator.setDuration(430);
            countAnimator.setInterpolator(CubicBezierInterpolator.DEFAULT);
        if (countLayout != null) {
            String oldStr = getStringOfCCount(currentCount);
            if (oldStr.length() == newStr.length()) {
                SpannableStringBuilder oldSpannableStr = new SpannableStri...
                SpannableStringBuilder newSpannableStr = new SpannableStri...
                SpannableStringBuilder stableStr = new SpannableStringBuil...
                for (int i = 0; i < oldStr.length(); i++) {
                    if (oldStr.charAt(i) == newStr.charAt(i)) {
                        oldSpannableStr.setSpan(new EmptyStubSpan(), i, i ...
                        newSpannableStr.setSpan(new EmptyStubSpan(), i, i ...
                    } else {
                        stableStr.setSpan(new EmptyStubSpan(), i, i + 1, 0);
                int countOldWidth = Math.max(AndroidUtilities.dp(12), (int...
                countOldLayout = new StaticLayout(oldSpannableStr, textPai...
                countAnimationStableLayout = new StaticLayout(stableStr, t...
                countAnimationInLayout = new StaticLayout(newSpannableStr,...
            } else {
                countOldLayout = countLayout;
        countWidthOld = countWidth;
        countAnimationIncrement = count > currentCount;
        countAnimator.start();
    if (count > 0) {
```

```
        countWidth = Math.max(AndroidUtilities.dp(12), (int) Math.ceil(tex...
        countLayout = new StaticLayout(newStr, textPaint, countWidth, Layo...
    currentCount = count;
    if (parent != null) {
        parent.invalidate();
private String getStringOfCCount(int count) {
    if (shortFormat) {
        return AndroidUtilities.formatWholeNumber(count, 0);
    return String.valueOf(count);
public void draw(Canvas canvas) {
    if (type != TYPE_CHAT_PULLING_DOWN && type != TYPE_CHAT_REACTIONS) {
        textPaint.setColor(ContextCompat.getColor(WKBaseApplication.getIns...
        circlePaint.setColor(ContextCompat.getColor(WKBaseApplication.getI...
    if (countChangeProgress != 1f) {
        if (animationType == ANIMATION_TYPE_IN || animationType == ANIMATI...
            updateX(countWidth);
            float cx = countLeft + countWidth / 2f;
            float cy = lastH / 2f;
            canvas.save();
            float progress = animationType == ANIMATION_TYPE_IN ? countCha...
            canvas.scale(progress, progress, cx, cy);
            drawInternal(canvas);
            canvas.restore();
        } else {
            float progressHalf = countChangeProgress * 2;
            if (progressHalf > 1f) {
                progressHalf = 1f;
            float countTop = (lastH - AndroidUtilities.dp(20)) / 2f;
            float countWidth;
            if (this.countWidth == this.countWidthOld) {
                countWidth = this.countWidth;
            } else {
                countWidth = this.countWidth * progressHalf + this.countWi...
            updateX(countWidth);
            float scale = 1f;
            if (countAnimationIncrement) {
                if (countChangeProgress <= 0.5f) {
                    scale += 0.1f * CubicBezierInterpolator.EASE_OUT.getIn...
                } else {
                    scale += 0.1f * CubicBezierInterpolator.EASE_IN.getInt...
            rectF.set(x, countTop, x + countWidth + AndroidUtilities.dp(11...
            canvas.save();
            canvas.scale(scale, scale, rectF.centerX(), rectF.centerY());
            if (drawBackground && circlePaint != null) {
                canvas.drawRoundRect(rectF, 11.5f * AndroidUtilities.densi...
                if (addServiceGradient) {
                    canvas.drawRoundRect(rectF, 11.5f * AndroidUtilities.d...
            canvas.clipRect(rectF);
            boolean increment = reverseAnimation != countAnimationIncrement;
            if (countAnimationInLayout != null) {
```

```
            canvas.save();
            canvas.translate(countLeft, countTop + AndroidUtilities.dp...
            textPaint.setAlpha((int) (255 * progressHalf));
            countAnimationInLayout.draw(canvas);
            canvas.restore();
        } else if (countLayout != null) {
            canvas.save();
            canvas.translate(countLeft, countTop + AndroidUtilities.dp...
            textPaint.setAlpha((int) (255 * progressHalf));
            countLayout.draw(canvas);
            canvas.restore();
        if (countOldLayout != null) {
            canvas.save();
            canvas.translate(countLeft, countTop + AndroidUtilities.dp...
            textPaint.setAlpha((int) (255 * (1f - progressHalf)));
            countOldLayout.draw(canvas);
            canvas.restore();
        if (countAnimationStableLayout != null) {
            canvas.save();
            canvas.translate(countLeft, countTop + AndroidUtilities.dp...
            textPaint.setAlpha(255);
            countAnimationStableLayout.draw(canvas);
            canvas.restore();
        textPaint.setAlpha(255);
        canvas.restore();
    } else {
        drawInternal(canvas);
public void updateBackgroundRect() {
    if (countChangeProgress != 1f) {
        if (animationType == ANIMATION_TYPE_IN || animationType == ANIMATI...
            updateX(countWidth);
            float countTop = (lastH - AndroidUtilities.dp(20)) / 2f;
            rectF.set(x, countTop, x + countWidth + AndroidUtilities.dp(11...
        } else {
            float progressHalf = countChangeProgress * 2;
            if (progressHalf > 1f) {
                progressHalf = 1f;
            float countTop = (lastH - AndroidUtilities.dp(20)) / 2f;
            float countWidth;
            if (this.countWidth == this.countWidthOld) {
                countWidth = this.countWidth;
            } else {
                countWidth = this.countWidth * progressHalf + this.countWi...
            updateX(countWidth);
            rectF.set(x, countTop, x + countWidth + AndroidUtilities.dp(11...
    } else {
        updateX(countWidth);
        float countTop = (lastH - AndroidUtilities.dp(20)) / 2f;
        rectF.set(x, countTop, x + countWidth + AndroidUtilities.dp(11), c...
private void updateX(float countWidth) {
```

```
        float padding = drawBackground ? AndroidUtilities.dp(5.5f) : 0f;
        if (gravity == Gravity.RIGHT) {
            countLeft = width - padding;
            if (horizontalPadding != 0) {
                countLeft -= Math.max(horizontalPadding + countWidth / 2f, cou...
            } else {
                countLeft -= countWidth;
        } else if (gravity == Gravity.LEFT) {
            countLeft = padding;
        } else {
            countLeft = (int) ((width - countWidth) / 2f);
        x = countLeft - padding;
    public float getCenterX() {
        updateX(countWidth);
        return countLeft + countWidth / 2f;
    public void setType(int type) {
        this.type = type;
    public void setParent(View parent) {
        this.parent = parent;
  public float getEnterProgress() {
    if (counterDrawable.countChangeProgress != 1f && (counterDrawable.animatio...
        if (counterDrawable.animationType == CounterDrawable.ANIMATION_TYPE_IN) {
            return counterDrawable.countChangeProgress;
        } else {
            return 1f - counterDrawable.countChangeProgress;
    } else {
        return counterDrawable.currentCount == 0 ? 0 : 1f;
  public boolean isInOutAnimation() {
    return counterDrawable.animationType == CounterDrawable.ANIMATION_TYPE_IN ...
public class RoundLayout extends FrameLayout {
  private static final int DEFAULT_CORNER = 5;
  private static final int DEFAULT_ALL_CORNER = Integer.MIN_VALUE;
  private int bgColor = Color.TRANSPARENT;
  private float allCorner = DEFAULT_ALL_CORNER;
  private float topLeftCorner = DEFAULT_CORNER;
  private float topRightCorner = DEFAULT_CORNER;
  private float bottomRightCorner = DEFAULT_CORNER;
  private float bottomLeftCorner = DEFAULT_CORNER;
  public RoundLayout(Context context) {
    super(context);
    setViewBackground();
  public RoundLayout(Context context, AttributeSet attrs) {
    super(context, attrs);
    extractAttribute(context, attrs);
    setViewBackground();
  public RoundLayout(Context context, AttributeSet attrs, int defStyleAttr) {
    super(context, attrs, defStyleAttr);
    extractAttribute(context, attrs);
    setViewBackground();
  public RoundLayout(Context context, AttributeSet attrs, int defStyleAttr, int ...
```

```
        super(context, attrs, defStyleAttr, defStyleRes);
        extractAttribute(context, attrs);
        setViewBackground();
    @Override
    protected void onLayout(boolean changed, int l, int t, int r, int b) {
        final int count = getChildCount();
        if (count > 1) {
            throw new IllegalStateException("View can have only single child");
        super.onLayout(changed, l, t, r, b);
    private void extractAttribute(Context context, AttributeSet attrs) {
        @SuppressLint("CustomViewStyleable") TypedArray ta = context.obtainStyledA...
        try {
            bgColor = ta.getColor(R.styleable.TextViewCorner_bgColor, Color.TRANSP...
            allCorner = ta.getDimension(R.styleable.TextViewCorner_allCorner, DEFA...
            topLeftCorner = ta.getDimension(R.styleable.TextViewCorner_topLeftCorn...
            topRightCorner = ta.getDimension(R.styleable.TextViewCorner_topRightCo...
            bottomRightCorner = ta.getDimension(R.styleable.TextViewCorner_bottomR...
            bottomLeftCorner = ta.getDimension(R.styleable.TextViewCorner_bottomLe...
        } finally {
            ta.recycle();
    public void setCorner(int all) {
        allCorner = all;
        setViewBackground();
    public void setCorner(int topLeft, int topRight, int bottomRight, int bottomLe...
        this.topLeftCorner = topLeft;
        this.topRightCorner = topRight;
        this.bottomRightCorner = bottomRight;
        this.bottomLeftCorner = bottomLeft;
        setViewBackground();
    public void setBgColor(int color) {
        bgColor = color;
        setViewBackground();
    private void setViewBackground() {
        Drawable drawable;
        if (allCorner > 0) {
            drawable = DrawableHelper.getCornerDrawable(
                    allCorner,
                    allCorner,
                    allCorner,
                    allCorner,
                    bgColor);
        } else {
            drawable = DrawableHelper.getCornerDrawable(
                    topLeftCorner,
                    topRightCorner,
                    bottomLeftCorner,
                    bottomRightCorner,
                    bgColor);
        DrawableHelper.setRoundBackground(this, drawable);
public class AnimatedFloat {
```

```
    private View parent;
    private Runnable invalidate;
    private float value;
    private float targetValue;
    private boolean firstSet;
    private long transitionDelay = 0;
    private long transitionDuration = 200;
    private TimeInterpolator transitionInterpolator = CubicBezierInterpolator.DEFA...
    private boolean transition;
    private long transitionStart;
    private float startValue;
    public AnimatedFloat() {
        this.parent = null;
        this.firstSet = true;
    public AnimatedFloat(long transitionDuration, TimeInterpolator transitionInter...
        this.parent = null;
        this.transitionDuration = transitionDuration;
        this.transitionInterpolator = transitionInterpolator;
        this.firstSet = true;
    public AnimatedFloat(long transitionDelay, long transitionDuration, TimeInterp...
        this.parent = null;
        this.transitionDelay = transitionDelay;
        this.transitionDuration = transitionDuration;
        this.transitionInterpolator = transitionInterpolator;
        this.firstSet = true;
    public AnimatedFloat(View parentToInvalidate) {
        this.parent = parentToInvalidate;
        this.firstSet = true;
    public AnimatedFloat(View parentToInvalidate, long transitionDuration, TimeInt...
        this.parent = parentToInvalidate;
        this.transitionDuration = transitionDuration;
        this.transitionInterpolator = transitionInterpolator;
        this.firstSet = true;
    public AnimatedFloat(View parentToInvalidate, long transitionDelay, long trans...
        this.parent = parentToInvalidate;
        this.transitionDelay = transitionDelay;
        this.transitionDuration = transitionDuration;
        this.transitionInterpolator = transitionInterpolator;
        this.firstSet = true;
    public AnimatedFloat(Runnable invalidate) {
        this.invalidate = invalidate;
        this.firstSet = true;
    public AnimatedFloat(Runnable invalidate, long transitionDuration, TimeInterpo...
        this.invalidate = invalidate;
        this.transitionDuration = transitionDuration;
        this.transitionInterpolator = transitionInterpolator;
        this.firstSet = true;
    public AnimatedFloat(Runnable invalidate, long transitionDelay, long transitio...
        this.invalidate = invalidate;
        this.transitionDelay = transitionDelay;
```